



Système de Surveillance Apicole Connecté

Rapport de Projet E3

ESIEE Paris – Année 2025/2026

Lauréat Jour des Projets 2026 – Prix de l'Innovation

Auteurs : AGUEH Thibault • AUCOUTURIER Charles • AZZA Anthony • BOURGOIS Ethan • BERDUGO Simon

Suiveur : FRANÇAIS Olivier (Département SEED)



Table des Matières

I.	Présentation du Contexte Apicole	3
I. A.	Marché du miel et consommation.....	3
I. B.	Structure du secteur apicole français	3
I. C.	La mortalité des colonies : un enjeu critique	4
II.	Projet Beetter	5
II. A.	Problématique et objectifs	5
II. B.	Organisation et management d'équipe.....	6
II. C.	Vue d'ensemble du projet	6
III.	Beetter Hive	10
III. A.	Conception matérielle.....	10
III. B.	Firmware	14
III. C.	Bilan énergétique et autonomie théorique.....	20
IV.	Beetter Home.....	22
IV. A.	Conception et architecture matérielle.....	22
IV. B.	Architecture logicielle et réseau.....	23
IV. C.	Architecture et organisation du code du tableau de bord web	24
IV. D.	Fonctionnalités du <i>dashboard</i>	26
IV. E.	Intelligence artificielle et classification comportementale	33
V.	Beetter Connect et application mobile Beetter	40
V. A.	Architecture du serveur distant.....	40
V. B.	API REST et sécurité	41
V. C.	Application mobile Beetter	43
V. D.	Déploiement et perspectives cloud	44
VI.	Bilan et Conclusion	46
VI. A.	Atteinte des objectifs fonctionnels.....	46
VI. B.	Valeur apportée & Business plan	46
VI. C.	Difficultés rencontrées.....	47
VI. D.	Perspectives d'évolution.....	52
VI. E.	Ressources numériques et démonstrateur.....	52
VI. F.	En conditions réelles chez un apiculteur	52
VII.	Annexes	57
VII. A.	Biologie apicole	57
VII. B.	Référentiel de monitoring	58
VII. C.	Acoustique apicole	60
VII. D.	L'Essaimage : phénomène biologique et enjeux de détection	62
VII. E.	Documentation projet	64
VIII.	Bibliographie.....	69
VIII. A.	Références	69
VIII. B.	Autres sources consultées	70
VIII. C.	Remerciements	70
VIII. D.	Mention IA	71



I. Présentation du Contexte Apicole

I. A. Marché du miel et consommation

La France consomme chaque année environ **45 000 tonnes** de miel^[1], soit près de **600 g par habitant**, mais sa production nationale n'en couvre qu'un tiers à peine : entre **15 000 et 30 000 tonnes** selon les saisons. Ce déséquilibre structurel contraint la filière à importer massivement, principalement d'Espagne, d'Ukraine et de Chine. Face à cette dépendance, la demande évolue : les consommateurs réclament davantage de traçabilité et de circuits courts, ce qui renforce la pression sur les producteurs locaux. Préserver les colonies et maximiser les rendements n'est donc plus seulement un impératif agronomique – c'est un enjeu économique et alimentaire de premier plan^[1].

I. B. Structure du secteur apicole français

La France compte aujourd'hui **57 500 apiculteurs** gérant plus de **1,45 million de colonies** d'abeilles réparties sur l'ensemble du territoire. Cette filière, vitale pour la biodiversité et la production agricole, est structurée de manière très hétérogène : **68 %** des apiculteurs possèdent moins de 10 ruches, tandis que seulement **3 %** en détiennent plus de 200. Cette fragmentation du secteur rend la surveillance individuelle des colonies particulièrement difficile à mettre en œuvre^{[1][2]}.

Indicateur	Valeur
Apiculteurs en France	57 500 recensés
Colonies en France	1 450 000 à 1 600 000 ruches
Petits exploitants	68 % < 10 ruches, 3 % > 200 ruches
Mortalité annuelle	25 % des colonies disparaissent chaque année
Durée de vie ouvrière	30 à 40 jours en été
Population par ruche	15 000 à 80 000 abeilles selon la saison
Ponte de la reine	Jusqu'à 2 000 œufs quotidiennement

Tableau I. B-1 Indicateurs clés de la filière apicole française (Annexe VII. A. Biologie Apicole)



I. C. La mortalité des colonies : un enjeu critique

Le taux de mortalité annuelle des colonies en France est estimé à 25 % des ruches chaque hiver – soit environ **350 000 ruches** perdues par an^[1] – et demeure préoccupant. Cette surmortalité alarmante constitue une menace directe pour la biodiversité et la production agricole. Loin d'être un accident isolé, ce déclin s'explique par la combinaison de plusieurs facteurs :

- Toxicité des pesticides : usage intensif de produits chimiques affectant le système nerveux des pollinisateurs ;
- Pression pathogène : propagation de parasites (*Varroa destructor*) et maladies (loque américaine) ;
- Stress climatique : altération des cycles de floraison réduisant les ressources trophiques ;
- L'absence d'outils de surveillance adaptés, en particulier pour les petits apiculteurs.

Ce contexte souligne la nécessité de concevoir des solutions technologiques permettant une détection précoce des anomalies, sans pour autant perturber le fonctionnement des colonies.

En résumé, le secteur apicole français souffre d'un manque d'outillage numérique adapté à la surveillance distante des ruches. Ce constat constitue le point de départ direct du projet Beetter.



II. Projet Beetter

Cette partie présente l'intégralité du travail effectué dans le cadre du projet Beetter.

II. A. Problématique et objectifs

L'apiculture traditionnelle impose des visites physiques régulières de chaque ruche, ce qui entraîne plusieurs problèmes majeurs :

- Chaque ouverture perturbe la colonie et génère un stress mesurable pour les abeilles, ce stress pouvant être thermique et/ou mécanique.
- La surveillance simultanée de plusieurs ruches dispersées géographiquement est impossible sans déplacement.
- Les anomalies critiques comme l'essaimage, les maladies ou la perte de reine sont souvent détectées trop tardivement pour être traitées efficacement.
- Les petits apiculteurs ne disposent pas toujours des ressources humaines ou matérielles pour des contrôles réguliers.

« **Comment permettre à un apiculteur de surveiller et de gérer une flotte de ruches à distance, sans intervention physique répétée, tout en préservant la santé des colonies ?** »

Beetter propose une solution pour pallier ces limites selon quatre objectifs structurels :

- **Monitoring non-intrusif** : acquisition continue de paramètres environnementaux et acoustiques.
- **Accessibilité économique** : coût unitaire à environ **200 €** (hors Beetter Home) contre plusieurs centaines d'euros pour des alternatives commerciales existantes comme Mellia (≈400€), Beep (Pays-Bas, ≈430€), Mellisphera (France, ≈350€ sans acoustique) et Arnia (Royaume-Uni, ≈500€), dont aucune ne propose l'intégralité des fonctionnalités Beetter à ce prix.
- **Centralisation et interopérabilité** : réception locale des mesures de Beetter Hive par Beetter Home, puis mise à disposition des utilisateurs sur Beetter.fr et l'application mobile Beetter.
- **Maintenance prédictive** : analyse spectrale pour la détection pré-événementielle (essaimage, intrusion, orphelinage).



II. B. Organisation et management d'équipe

Le projet a été coordonné par Charles Aucouturier en tant que chef de projet, secondé par Ethan Bourgois sur l'ensemble des volets matériels et organisationnels. Le développement repose sur cinq pôles fonctionnels :

- **Organisation et Livrables** : (Charles & Ethan) Planification des tâches, coordination de l'écosystème (Beetter Hive, Beetter Home, Beetter.fr), gestion budgétaire, achat des composants, création des livrables (rapport de Projet, rapport éthique, affiche, vidéo, et soutenance orale).
- **Hardware** : (Charles & Ethan) Conception du PCB, sélection des composants, intégration de l'architecture énergétique, assemblage de Beetter Hive et Beetter Home.
- **Firmware** : (Charles & Ethan) Programmation bas niveau (ESP32-C6), optimisation énergétique, traitement initial I2S, protocole LoRa, communication SPI, UART, I2C.
- **Software** : (Thibault & Anthony) Architecture logicielle globale, configuration de Beetter Home (réception LoRa, InfluxDB/PostgreSQL), *backend* cloud (VPS, API REST), développement de Beetter.fr et de l'application Beetter.
- **IA & Algorithme** : (Anthony & Simon) Constitution du référentiel acoustique, entraînement des modèles d'analyse FFT et MFCC pour la classification des états.

Dès le lancement du projet, Charles et Ethan ont mis en place une infrastructure de travail collaboratif structurée autour de trois outils.

Un serveur Discord dédié et un Google Drive partagé ont été organisés selon les mêmes pôles que l'équipe : 0. Documents, 1. Livrables, 2. Hardware, 3. Firmware, 4. Software, 5. IA. Cette cohérence entre l'organisation humaine et l'organisation des ressources a facilité le suivi et les échanges entre membres.

Le Drive héberge notamment le Beetter Board, document central qui regroupe la liste des composants, un tableau de suivi des tâches (statut, date d'échéance, étapes, priorité et responsable), un tableau des objectifs fonctionnels détaillant les cibles par domaine (Firmware, hardware, software, IA), ainsi qu'un journal des réunions avec notre enseignant-suiveur Olivier Français. Les consignes et documents institutionnels transmis par l'ESIEE y sont également archivés.

GitHub a servi de dépôt de référence pour l'ensemble du code source et de la documentation technique, permettant la traçabilité des modifications et l'archivage des versions successives du projet. Le dépôt est accessible à l'adresse : <https://github.com/Charlox29/Beetter>.

II. C. Vue d'ensemble du projet

L'écosystème Beetter se compose de deux modules matériels complémentaires :

- **Beetter Hive** (de l'anglais « ruche »), boîtier embarqué installé à l'intérieur de la ruche. Il intègre l'ensemble des capteurs et assure l'acquisition continue des données environnementales (température, humidité, luminosité) et acoustiques (fréquence dominante, amplitude RMS, 13 coefficients MFCC). Conçu au format d'un cadre Dadant standard, il s'installe sans modification de la ruche et émet une trame binaire de 100 octets toutes les 30 secondes via LoRa.
- **Beetter Home** (de l'anglais « maison »), la passerelle locale installée chez l'apiculteur. Elle reçoit les trames des boîtiers Hive via LoRa, les décode, les stocke localement et les transmet vers le cloud, assurant ainsi la continuité du service même en zone sans couverture internet.

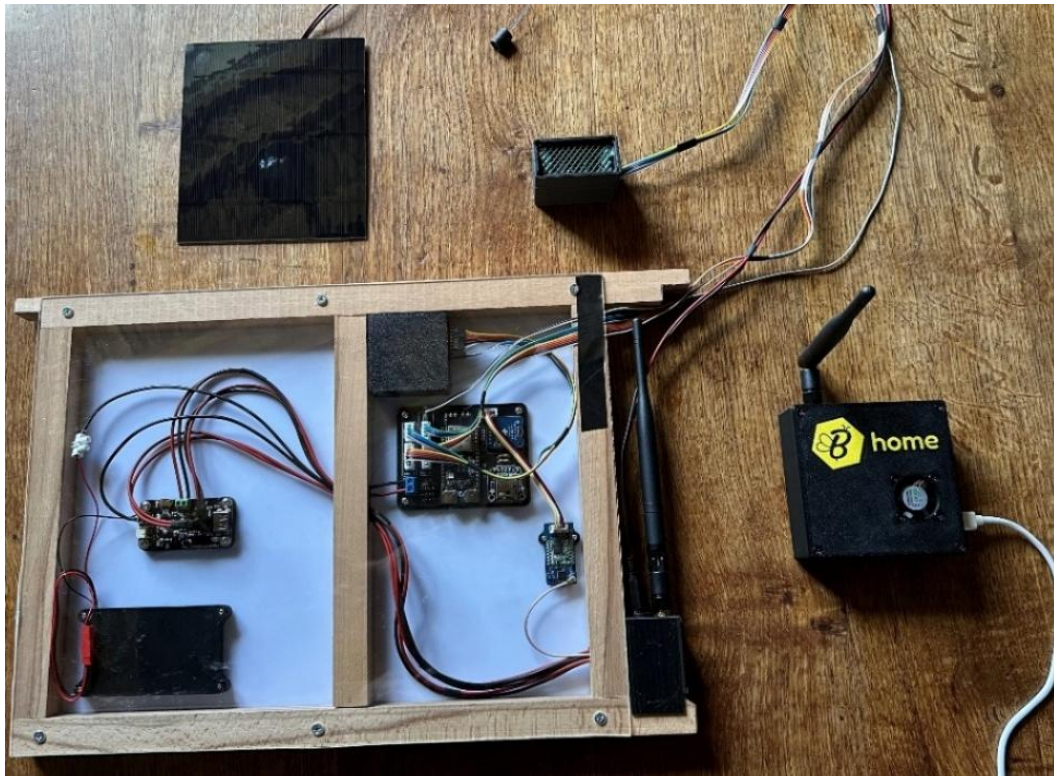


Figure II. C-1 Module Beetter Hive à gauche et Beetter Home à droite

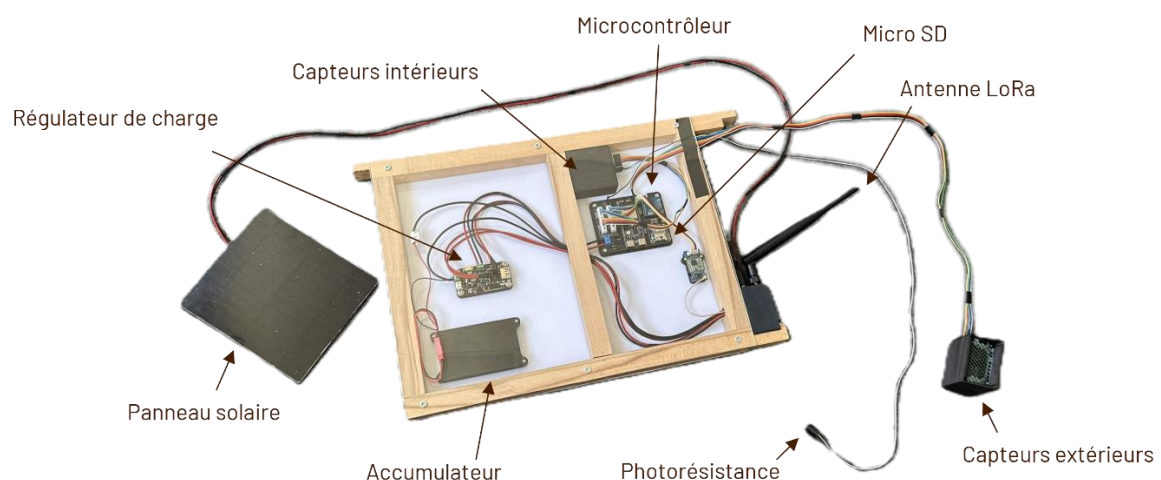


Figure II. C-2 Schéma composants Beetter Hive



Ces deux module s'inscrivent dans une architecture de bout en bout organisée en quatre couches successives :

1. **Acquisition (Beetter Hive) :** Le boîtier capte en continu la température et l'humidité intérieures et extérieures, la luminosité et le son. Le Firmware calcule les 13 coefficients MFCC localement sur l'ESP32-C6, puis sérialise l'ensemble en une trame binaire de 100 octets (blocs ENV + AUD) émise toutes les 30 secondes à 868 MHz. Le *duty cycle* mesuré est de 0,60 %, bien en deçà de la limite réglementaire ETSI de 1 %.
2. **Passerelle locale (Beetter Home) :** Le Raspberry Pi reçoit les trames via le module Grove LoRa, les décode et les injecte dans la base de données locale : InfluxDB pour les séries temporelles de mesures, PostgreSQL pour les métadonnées (ruches, utilisateurs, alertes). Un premier niveau de traitement par intelligence artificielle est effectué localement à cette étape (voir IV.E). En cas de coupure internet, la collecte se poursuit sans interruption et les données sont synchronisées dès le rétablissement de la connexion.
3. **Infrastructure cloud (VPS) :** Un serveur distant agrège les données de plusieurs Beetter Home, expose une API REST sécurisée (JWT) et assure la réplication multi-ruchers. C'est le point d'entrée unique pour l'application mobile et les tableaux de bord distants.
4. **Restitution (dashboard web et application mobile) :** L'apiculteur accède à ses données via le *dashboard* web local (hébergé sur Beetter Home), le *dashboard* centralisé (sur le VPS) et l'application mobile Beetter. Ces interfaces affichent les mesures en temps réel, l'historique navigable, le journal des alertes et les prédictions comportementales issues du modèle IA.

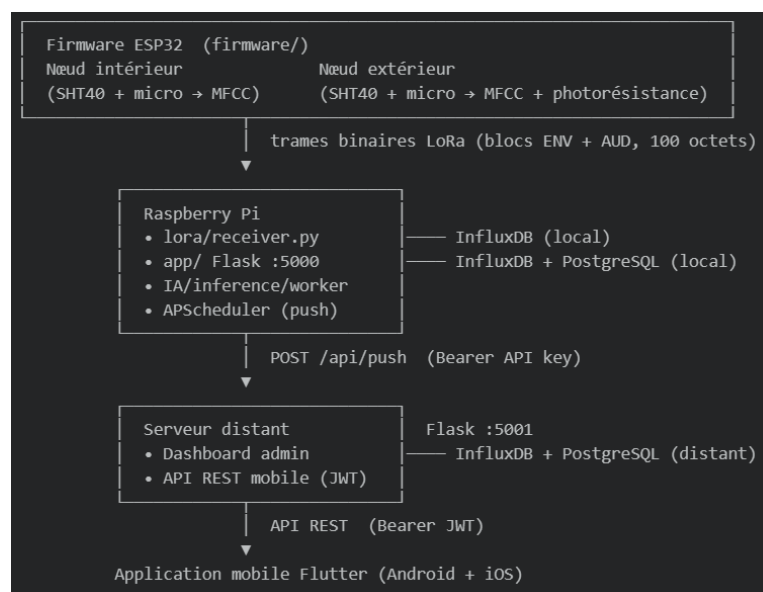


Figure II. C-3 Architecture réseau et flux de données de l'écosystème Beetter

L'architecture ainsi définie assure une résilience maximale : en cas de coupure internet, les données continuent d'être collectées localement, puis synchronisées dès le rétablissement de la connexion.



Couche locale (Edge)	Couche cloud (VPS)
• Réception LoRa (N ruches) • InfluxDB (time-series) • PostgreSQL (métadonnées) • <i>Dashboard</i> Flask local • Traitement IA local	• Agrégation multi-ruchers • API REST tierce • <i>Dashboard</i> centralisé • Alertes <i>smartphone</i>

Tableau II. C-1 Synthèse des couches locales (Edge) et cloud (VPS) de l'architecture



III. Beetter Hive

III. A. Conception matérielle

III.A.1. Choix des composants

Le module Beetter Hive s'intègre dans la ruche et s'articule autour d'un microcontrôleur **ESP32** intégré sur un circuit imprimé (PCB) conçu en interne. Le choix de celui-ci résulte d'un compromis étudié entre puissance de calcul, consommation énergétique, connectivité et coût. Le tableau ci-dessous détaille les composants retenus et leur rôle dans l'architecture.

Composant	Rôle
ESP32-C6-DevKitC-1-N8	Unité centrale de traitement. Gère l'acquisition des données, l'analyse et la transmission.
SEN0428 - SHT40 (x2)	Capteur de température et d'humidité. Mesures en continu de l'environnement intérieur et extérieur de la ruche.
ADA6049 - ICS-43434 (x2)	Microphone numérique I2S dédié à l'acquisition acoustique. Permet l'analyse spectrale par FFT/MFCC des bourdonnements.
RFM95 - Grove LoRa 113060006	Module radio Long Range 868MHz. Transmission des données sur plusieurs kilomètres entre Beetter Hive et Beetter Home.
ADA3295 - Horloge RTC I2C PCF8523	Horloge Temps Réel. Horodatage autonome des mesures, synchronisé via internet lors de la reconnexion.
ADA4682 - Lecteur Micro SD	Stockage local des données en cas de perte de connectivité LoRa.
GL5539 - Photorésistance 10kΩ	Capteur de luminosité. Mesure de l'intensité lumineuse extérieure pour contextualiser les données (cycle jour/nuit, météo).
DFR0559 - Solar Power Manager 5V	Module Sunflower de gestion du cycle charge/décharge de la batterie LiPo et du panneau solaire SOL3W.
Panneau Solaire SOL3W	Un panneau solaire Seeed Studio SOL3W (138X160mm) qui recharge en continu la batterie.
L805080 - Accumulateur LiPo	Un accumulateur LiPo 3,7V / 4000 mAh pour le stockage d'énergie.

Tableau III. A-1 Composants matériels du module Beetter Hive et leur rôle

III.A.2. Conception du PCB

La conception du PCB a été réalisée sous KiCad, logiciel de conception électronique libre qui prend en charge la schématique, le placement des composants et le routage des pistes.

La première étape a consisté à constituer une bibliothèque fiable. Pour les composants disponibles en ligne – notamment les circuits intégrés courants (ESP32-C6, RTC) – Les empreintes ont été importées depuis des dépôts officiels (SnapEDA). Pour les composants non référencés ou dont les empreintes disponibles ne correspondaient pas exactement à nos besoins, Les empreintes ont été réalisées manuellement : en reprenant la base des connecteurs Dupont 2,54mm et des connecteurs JST XH depuis la bibliothèque KiCad, puis en dessinant de nouveaux contours selon les *datasheets* constructeur des composants.

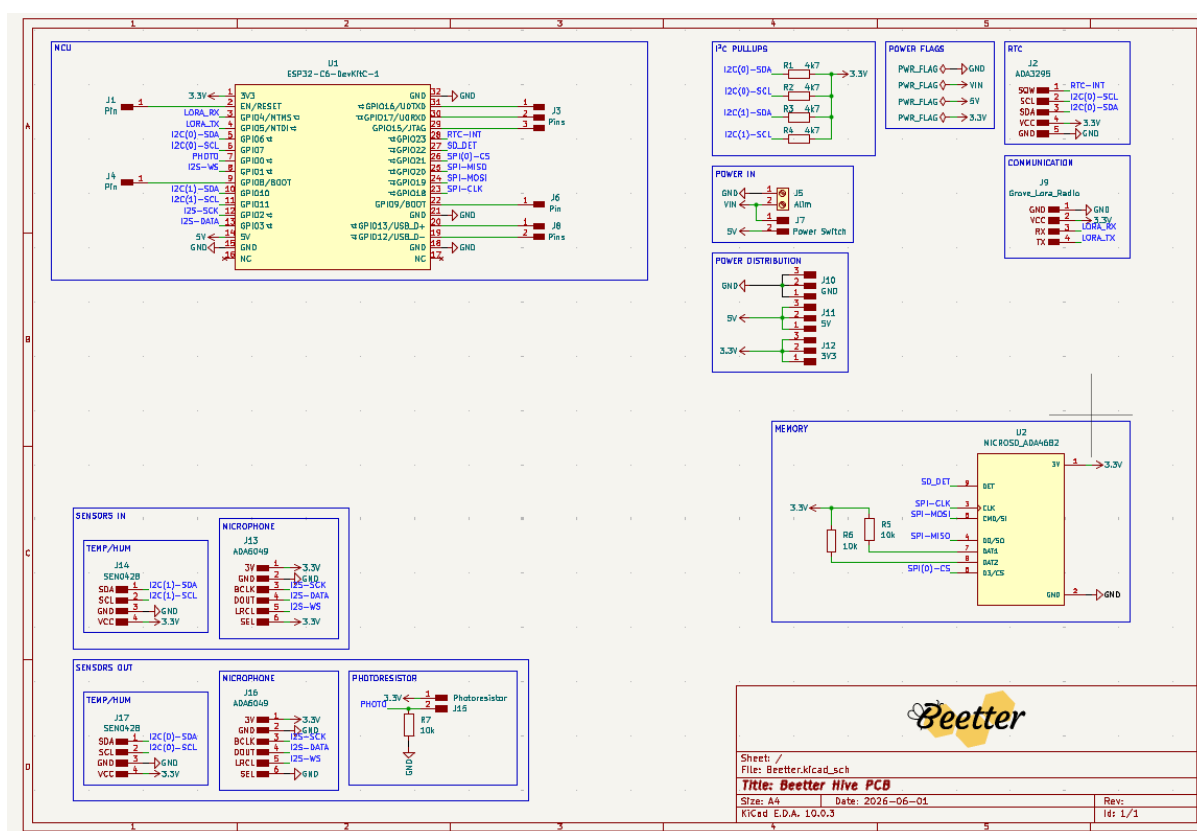


Figure III. A-1 Schéma électronique du PCB sous KiCad

Le routage a privilégié la compacité – afin, entre autres, de respecter une hauteur maximum imposée par la compatibilité Dadant – tout en adaptant les largeurs de pistes entre celles dédiées aux données et celles dédiées à l'alimentation.

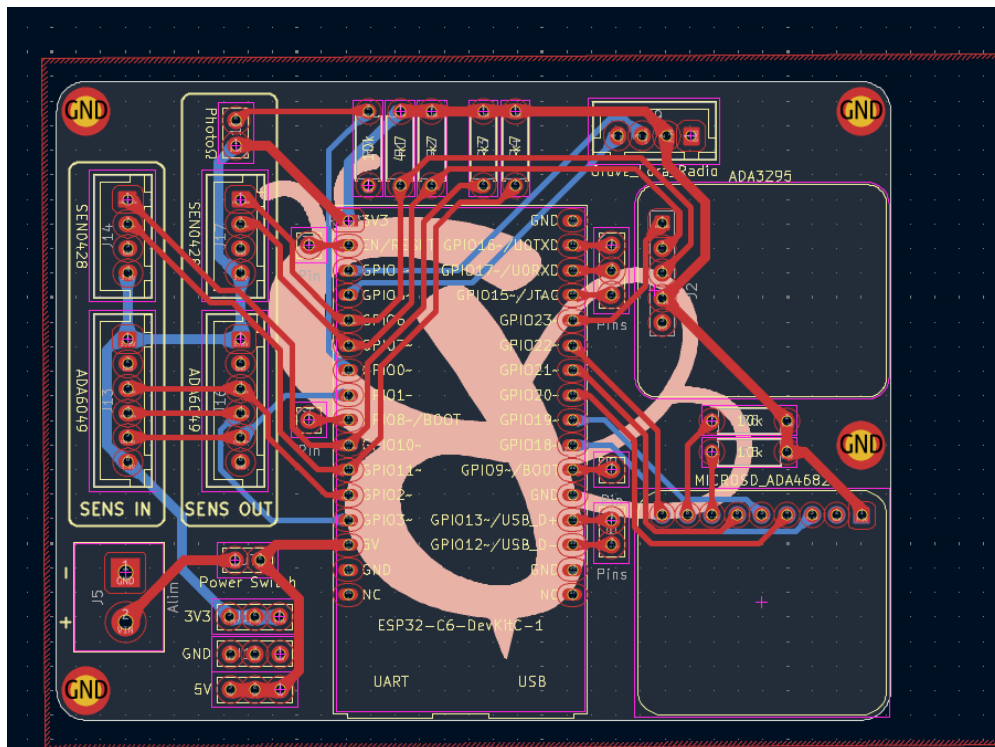


Figure III. A-2 Design PCB sous KiCad

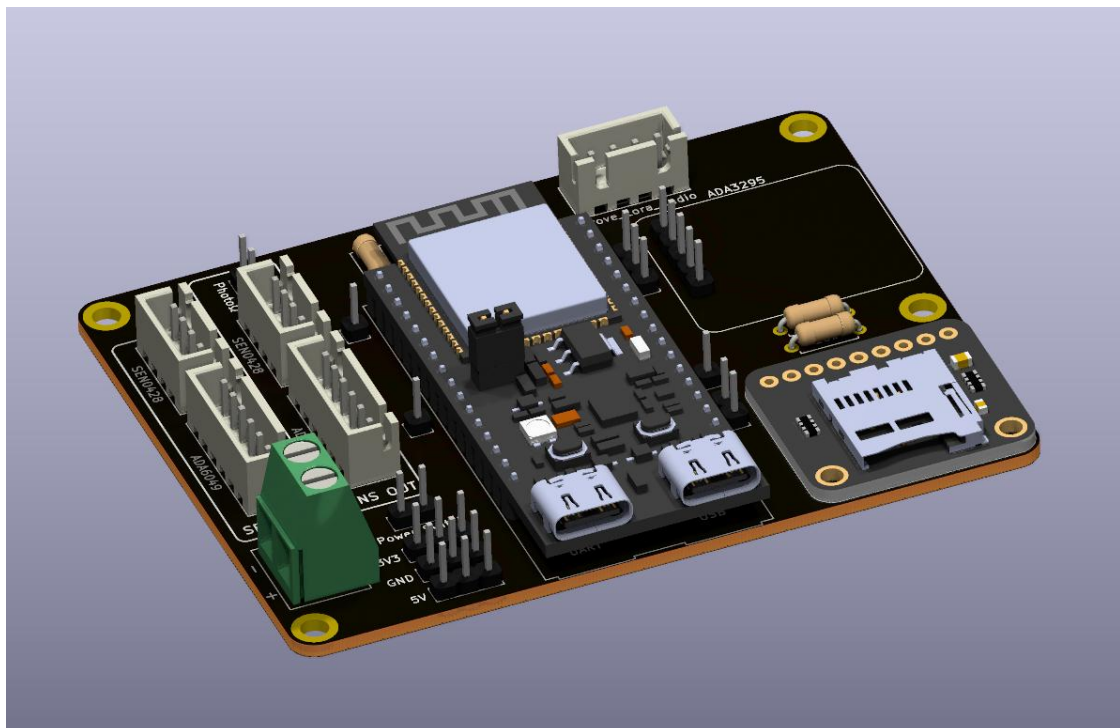


Figure III. A-3 Rendu 3D du PCB sous KiCad

III.A.3. Conception mécanique et boîtier

La conception mécanique a été réalisée sous Autodesk Fusion 360. Beetter Hive adopte une conception hybride : le châssis principal s'inspire des cotes normalisées d'un cadre Dadant standard (435 x 300 mm avec épaulements), dont l'armature a été construite avec un tasseau de

bois massif de 22 mm de côté et les faces comblées avec des plaques de polystyrène de 2,5 mm d'épaisseur (une opaque et une transparente). Ce cadre-support – accueillant les éléments électroniques vissés à des pièces imprimées en 3D collées – assure à la fois l'isolation et le maintien des composants sans visserie intrusive dans la ruche. Le panneau solaire est intégré sur la face supérieure du cadre. Les capteurs extérieurs sont logés dans un boîtier à part relié par des nappes électriques au Beetter Hive.

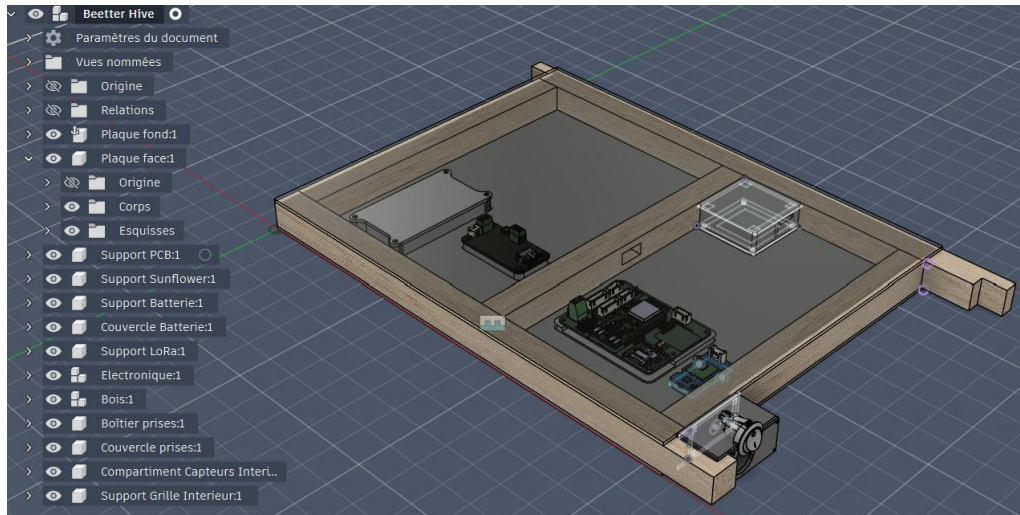


Figure III. A-4 Modélisation Beetter Hive sur Fusion 360

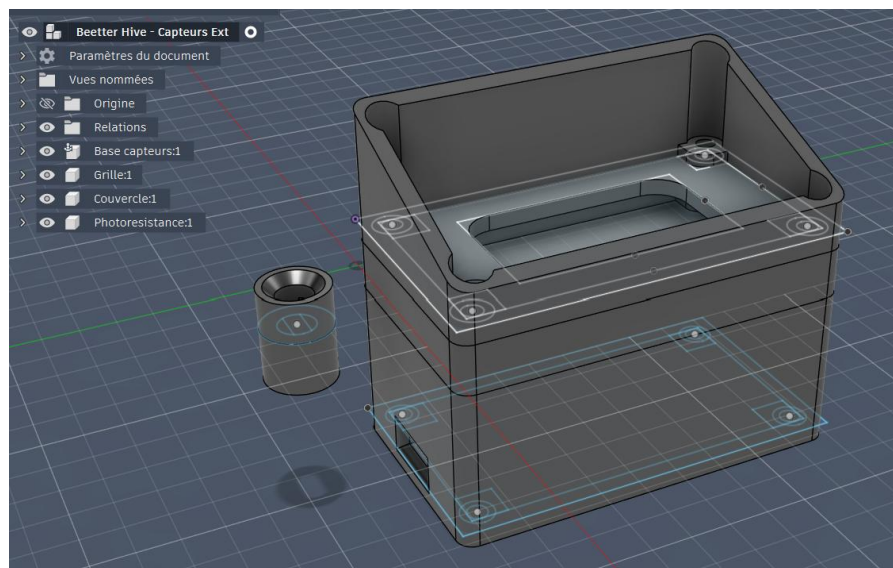


Figure III. A-5 Modélisation du boîtier extérieur et du support de photorésistance de Beetter Hive sur Fusion 360

Toutes les pièces imprimées en 3D ont été produites à partir de PETG ; ce matériau est moins courant que le PLA mais tolère mieux des contraintes de chaleur.



III.A.4. Alimentation et autonomie énergétique

L'autonomie énergétique constitue un critère déterminant : le système doit fonctionner dans des environnements sans prise de courant, potentiellement isolés. Une alimentation propre a été choisie pour chaque boîtier : une batterie, un panneau solaire, et un module gérant la charge, la décharge, et l'alimentation de notre système. Le dimensionnement de ces composants a été surévalué au début pour assurer un fonctionnement continu même en cas de faible ensoleillement prolongé.

III. B. Firmware

Cette section détaille l'implémentation embarquée du module Beetter Hive : cycle d'exécution du Firmware, comportement de chaque capteur, pipeline de traitement MFCC et format binaire de la trame LoRa transmise toutes les 30 secondes à Beetter Home.

III.B.1. Architecture logicielle et cycle d'exécution

Le Firmware s'exécute dans une boucle principale (*loop()*) dont chaque itération dure exactement 30 secondes. Les mesures actives occupent environ 7 secondes ; le reste est de l'attente.

Etape	Durée typique	Opération	Détail
1	20 ms	SHT40 INT + EXT	T°, %RH, humidité absolue - deux bus I2C distincts
2	5 ms	Photorésistance GL5539	ADC brut → indice logarithmique 0,0–10,0
3	3 050 ms	MFCC micro intérieur	Capture I2S 3 s + 11 fenêtres × 2 048 samples
4	3 050 ms	MFCC micro extérieur	Idem, canal droit du bus I2S stéréo
5	15 ms	Écriture SD	3 fichiers CSV – 1 seule ouverture/fermeture
6	176 ms	Envoi LoRa	Trame ENV+AUD 100 bytes, <i>duty cycle</i> 0,60 %
7	15 000 ms	[1/20 cycles] WAV stéréo	Enregistré pendant l'attente – transparent
8	20 s (reste)	Attente	Jusqu'à t = 30 s depuis le début du cycle

Tableau III. B-1 Étapes du cycle séquentiel du Firmware



Un *watchdog* (*Task Watchdog Timer*, *esp_task_wdt*), adossé à un *timer* matériel de l'ESP32-C6, est armé à 60 secondes dès le *setup()* et réarmé à chaque itération de la boucle. En cas de blocage du bus I2S, d'une SD corrompue ou de tout autre verrouillage, le compteur n'est plus rafraîchi : la puce redémarre alors automatiquement, sans intervention humaine, un comportement critique pour un déploiement en milieu naturel.

Mise en veille (optimisation envisagée, non implémentée à ce stade). Pendant la phase d'attente du cycle (environ 20 s sur 30, soit le poste de consommation dominant) l'ESP32-C6 reste actif via un simple *delay()* et consomme de l'ordre de 20 mA. Deux modes de veille permettraient de réduire cette consommation :

- **Light sleep** : le CPU est suspendu mais la RAM et l'état des périphériques sont conservés. Le *buffer* de capture statique de 94 KB survit, et le réveil sur *timer* se fait en quelques millisecondes : la consommation d'attente chuterait d'environ 20 mA à ~1 mA, réduisant nettement la moyenne du cycle sans modifier l'architecture du Firmware. C'est l'optimisation la plus directe et la plus rentable.
- **Deep sleep** : plus agressif (~10-20 μ A), il coupe tout sauf le domaine RTC mais efface la RAM. Il imposerait une réinitialisation complète des périphériques (I2S, LoRa, SD) à chaque réveil, repartirait du *setup()* à chaque cycle et serait incompatible avec l'enregistrement WAV continu : il ne deviendrait pertinent qu'en allongeant sensiblement l'intervalle entre deux cycles.

Dans les deux cas, le réveil devra être cadencé avant l'échéance du *watchdog* (ou ce dernier nourri au réveil) afin d'éviter un redémarrage intempestif – veille et *watchdog* participant tous deux du même objectif d'autonomie en fonctionnement non surveillé.

III.B.2. Acquisition des capteurs environnementaux

Deux capteurs SHT40 (SEN0428) sont interrogés via I2C à chaque cycle. Le capteur extérieur utilise le bus I2C matériel (GPIO6/7, partagé avec le RTC) et le capteur intérieur utilise un bus I2C logiciel *bit-bang* (GPIO10/11) pour éviter le conflit d'adresse 0x44. L'humidité absolue (g/m^3) est calculée localement via Magnus-Tetens et stockée dans les CSV SD.

Champ transmis	Encodage Trame	Résolution	Plage
temp_int / temp_ext	int16 = $^{\circ}\text{C} \times 100$	0,01 $^{\circ}\text{C}$	-327 à +327 $^{\circ}\text{C}$
hum_int / hum_ext	uint16 = %RH $\times 10$	0,1 %RH	0 à 100 %RH

Tableau III. B-2 Encodage des mesures SHT40 dans la trame LoRa



III.B.3. Pipeline de traitement acoustique MFCC

Deux microphones ICS-43434 partagent le même bus I2S stéréo. Le micro extérieur occupe le canal gauche (SEL=GND), le micro intérieur le canal droit (SEL=3,3V). La FFT (Fast Fourier Transform) est une étape intermédiaire du calcul des MFCC (Mel-Frequency Cepstral Coefficients), et non un descripteur transmis indépendamment.

Etape	Opération	Détail technique
1	Capture I2S	3 s de son à 8 000 Hz (fréquence d'échantillonnage) = 24 000 échantillons sur 24 bits, rangés dans un <i>buffer</i> fixe de 94 KB (alloué une fois au démarrage, jamais fragmenté)
2	Découpage fenêtres	Le signal échantillonné est ensuite découpé en 11 tranches consécutives (« fenêtres ») de 2 048 échantillons, soit 256 ms chacune, sans recouvrement
3	RMS par fenêtre	Les échantillons entiers sont convertis en flottants, puis on calcule la valeur efficace (RMS, l'amplitude moyenne du son) de la fenêtre, mise de côté pour le moyennage
4	Fenêtrage Hamming	Chaque échantillon de la fenêtre est multiplié par un coefficient donné par $w(n) = 0,54 - 0,46 \cdot \cos(2\pi n / (N-1))$, dite « fenêtrage de Hamming » pour atténuer les bords de la tranche pour limiter les fuites spectrales (artefacts) de la FFT
5	FFT	Transformée de Fourier (arduinoFFT 2.0) sur $N = 2\,048$ points : le spectre est découpé en « bins », des cases de fréquence de 3,9 Hz de large ($8\,000 / 2\,048$)
6	Fréquence dominante	On repère le <i>bin</i> de plus forte amplitude dans la bande 50-2 000 Hz (la plage où se situent les signatures acoustiques de la ruche), sa fréquence devient la fréquence dominante de la tranche, c'est-à-dire la tonalité la plus présente, mise de côté pour le moyennage
7	26 filtres Mel	Le spectre complet est regroupé par 26 filtres triangulaires espacés selon l'échelle Mel (qui imite la perception de l'oreille), entre 80 et 4 000 Hz ; on prend le logarithme de l'énergie de chaque filtre
8	DCT-II	Une transformée en cosinus discrète (DCT-II) condense les 26 log-énergies en 13 coefficients MFCC, mis de côté pour le moyennage. Ce sont ces 13 nombres qui constituent la « signature » du son utilisée ensuite par le modèle d'intelligence artificielle
9	Moyennage	Les 11 tranches ont chacune produit un RMS, une fréquence dominante et 13 coefficients MFCC (étapes 3, 6 et 8). On fait la moyenne de ces 11 valeurs pour chaque grandeur : le résultat final est un seul jeu de valeurs par microphone, représentatif d'environ 2,8 s de signal (11 tranches $\times$ 256 ms).

Tableau III. B-3 Pipeline de traitement acoustique MFCC



Rappel : un *bin* est une case de fréquence de 3,9 Hz issue de la FFT. Pour chaque fenêtre de 2 048 échantillons, le Firmware repère le *bin* de magnitude maximale dans la bande 50-2000 Hz et l'accumule. Après les 11 fenêtres, la somme est divisée par 11 (chaque fenêtre pesant identiquement, soit 256 ms) : la fréquence transmise reflète la tonalité la plus présente sur les $\approx 2,8$ s analysées, encodée en $\text{Hz} \times 10$ (résolution 0,1 Hz). De la même façon, la valeur RMS transmise est la moyenne des 11 valeurs RMS - une amplitude efficace par fenêtre, et non une énergie - encodée en $\text{RMS} \times 10\,000$ (résolution 0,0001). Ce découpage en fenêtres moyennées reproduit le comportement par défaut de *librosa.feature.mfcc()*, ce qui garantit la cohérence entre les descripteurs calculés à bord et ceux utilisés pour l'entraînement du modèle ML.

La plage analysée (50 - 2 000 Hz) couvre l'intégralité des signatures acoustiques apicoles documentées dans la littérature^{[4][5][6]} :

- Colonie calme (butinage, ventilation) : 200 – 260 Hz^{[4][7]}
- Stress léger, manque de ressources : 300 – 400 Hz^{[5][7]}
- Essaimage imminent / chant de la reine vierge : 350 – 500 Hz^{[8][9][10]}
- Orphelinage, colonie anxieuse : 300 – 450 Hz^{[5][6]}
- Intrusion frelon asiatique : 400 – 600 Hz^[11]
- Harmoniques H2-H8 de l'abeille domestique : 460 – 1 840 Hz^[4]

III.B.4. Sérialisation et émission LoRa

Toutes les 30 secondes, le Firmware émet une trame binaire unique de 100 bytes à 868 MHz (fréquence réglementaire française), composée de deux blocs concaténés. Le *time-on-air* mesuré est de 176 ms, soit un *duty cycle* de 0,60 %, bien en deçà de la limite ETSI EN 300 220 de 1 % sur la sous-bande g1 (European Telecommunications Standards Institute)

Position	Taille	Bloc	Magic
bytes 0 - 24	25 bytes	BLOC ENV - données environnementales	0xE0
bytes 25 - 99	75 bytes	BLOC AUD - données acoustiques	0xA0

Tableau III. B-4 Vue d'ensemble de la trame LoRa

Offset	Bytes	Champ	Type	Encodage
0	1	magic	uint8	0xE0 (fixe)
1	4	seq	uint32	Compteur de trames – overflow à ~4 milliards de cycles
5	4	hive_id	char[4]	ASCII, ex. « B001 »



Offset	Bytes	Champ	Type	Encodage
9	4	timestamp	uint32	Unix UTC (secondes) - RTC PCF8523
13	2	temp_int	int16	°C × 100 (ex. 3412 → 34,12 °C)
15	2	hum_int	uint16	%RH × 10 (ex. 620 → 62,0 %)
17	2	temp_ext	int16	°C × 100
19	2	hum_ext	uint16	%RH × 10
21	2	light_ext	uint16	indice × 10
23	2	crc16	uint16	CRC-16/CCITT

Tableau III. B-5 BLOC ENV - 0xE0 - 25 bytes

Offset	Bytes	Champ	Type	Encodage
0	1	magic	uint8	0xA0 (fixe)
1	4	seq	uint32	Même compteur que le BLOC ENV correspondant
5	4	hive_id	char[4]	ASCII, ex. « B001 »
9	4	timestamp	uint32	Unix UTC (secondes)
13	2	freq_int	uint16	Hz × 10 - fréquence dominante int. (moy. 11 fenêtres)
15	2	rms_int	uint16	RMS × 10 000 - énergie acoustique moy. sur 2,8 s
17	2	freq_ext	uint16	Hz × 10 - micro extérieur
19	2	rms_ext	uint16	RMS × 10 000 - micro extérieur
21	26	mfcc_int[13]	int16[13]	MFCC0 × 10 (résol. 0,1) ; MFCC1..12 × 1 000 (résol. 0,001)
47	26	mfcc_ext[13]	int16[13]	Même encodage - micro extérieur
73	2	crc16	uint16	CRC-16/CCITT (poly 0x1021, init 0xFFFF)

Tableau III. B-6 BLOC AUD – 0xA0 – 75 bytes

III.B.5. Fiabilité et redondance des données

Pour garantir l'intégrité des données collectées, plusieurs mécanismes de redondance ont été intégrés dès la conception :

- Stockage local sur Micro SD en cas de perte de la connexion LoRa ;



- Horodatage autonome via le module RTC, indépendant de toute connectivité ;
- Synchronisation automatique de l'horloge dès la reconnexion à internet ;
- Transmission des données accumulées lors du rétablissement de la connexion.

III.B.6. Stockage local sur carte micro-SD

En parallèle de l'envoi LoRa, chaque cycle produit trois fichiers CSV sur la micro-SD (ADA4682, bus SPI). Les trois fichiers sont ouverts et fermés en une seule opération par cycle (*beginCycle/endCycle*). L'en-tête est recréé automatiquement si le fichier est absent ou vide.

Fichier	En-tête CSV	Volume	Autonomie (avec 16 Go de stockage)
/env.csv	timestamp, T_int_C, H_int_RH, Ha_int_gm3, T_ext_C, H_ext_RH, Ha_ext_gm3, lum_brut, lum_indice	70 bytes/ligne	~44 ans
/mfcc_int.csv	timestamp, freq_dom_Hz, rms, mfcc0..mfcc12	135 bytes/ligne	~44 ans
/mfcc_ext.csv	timestamp, freq_dom_Hz, rms, mfcc0..mfcc12	135 bytes/ligne	~44 ans
WAV stéréo	PCM 16 bits, 8 000 Hz - 1 clip 15 s toutes les 10 min	470 KB/clip	249 jours soit 8 mois de stockage, c'est le facteur limitant

Tableau III. B-7 Fichiers produits sur la carte micro-SD

Les trois flux CSV partagent la carte SD avec les clips WAV, qui représentent à eux seuls 98 % du volume écrit : la durée de remplissage réelle d'une carte de 16 Go est donc entièrement dictée par le WAV, soit environ 8 mois, les CSV restant négligeables. Ce point a été vérifié sur le terrain : après 6 jours de déploiement, la carte contenait 227 Mo, dont 223 Mo de WAV (464 clips) et moins de 4 Mo de CSV (moins de relevés prévus à cause de pannes décrites en VI.F.). Pour allonger l'autonomie de stockage sans changer de carte, les leviers sont l'espacement des clips (paramètre WAV_EVERY_CYCLES) ou leur durée (WAV_DUREE_SEC) sinon il faut opter pour une carte de plus grande capacité.

Tous les horodatages produits par le module (noms de fichiers WAV, colonne timestamp des CSV, dates de modification FAT) sont exprimés en UTC : le RTC stocke l'heure UTC, à convertir en heure locale (UTC+2 en été) lors de l'analyse.



III.B.7. Fichier de configuration centralisé

Le Firmware concentre l'ensemble des paramètres modifiables (identifiant de ruche, brochage GPIO, fréquence LoRa, paramètres de calibration de la photorésistance, fenêtre d'analyse acoustique, cadence des clips WAV, fuseau horaire de compilation, timeout du *watchdog*...) dans un unique fichier d'en-tête *BeetterConfig.h*, séparé du code métier. Cette centralisation permet de reconfigurer un boîtier (changement de ruche, ajustement du brochage en cas de révision PCB, calibration terrain) sans modifier la logique applicative, réduisant le risque de régression lors d'un déploiement en flotte.

Au-delà des paramètres, chaque composant matériel est piloté par un module dédié (*BeetterLoRa*, *BeetterMic*, *BeetterRTC*, *BeetterSHT40*, *BeetterSD*...) qui s'appuie sur une bibliothèque open-source éprouvée plutôt que sur du code bas niveau réécrit.

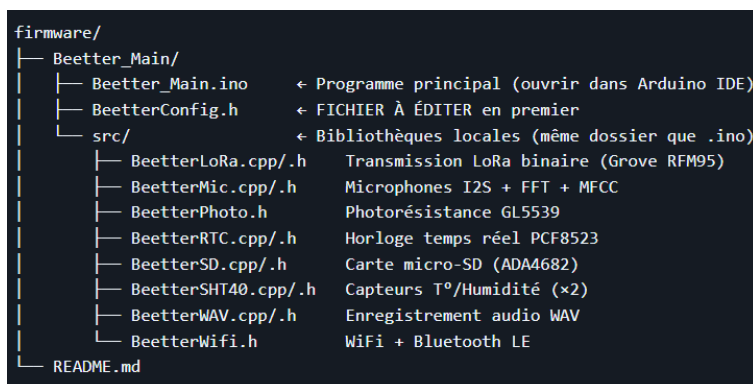


Figure III. B-1 Arborescence du Firmware/

En synthèse, le Firmware *Beetter Hive* est conçu pour une exécution déterministe et autonome : chaque mesure est effectuée, stockée localement et transmise dans un cycle rigide de 30 secondes, sans dépendance au moniteur série ni à une connexion réseau active. La conception matérielle privilégie la robustesse, la redondance et l'autonomie, trois critères essentiels pour un déploiement en milieu naturel et sans surveillance humaine constante.

III. C. Bilan énergétique et autonomie théorique

L'alimentation repose sur une batterie LiPo 3,7 V / 4 000 mAh rechargée par le panneau solaire SOL3W (3 W crête, 138 × 160 mm) via le régulateur de charge DFR0559. *Beetter Hive* est conçu pour une autonomie totale 24 h/24, 7 j/7 sans accès au réseau électrique.

La consommation moyenne est d'environ 30 mA, soit 720 mAh/j (2,66 Wh/j), répartie sur le cycle de 30 secondes :

- Idle / attente : 20 mA × 20 s
- Captures I2S MFCC × 2 : 55 mA × 6,1 s
- Envoi LoRa : 120 mA × 0,18 s
- SD + SHT40 + LDR : 80 mA × 0,09 s



- WAV ($\div 20$ cycles) : $85 \text{ mA} \times 0,75 \text{ s}$ équivalent

Méthode de calcul : l'apport solaire est estimé par $\text{Apport (Wh/j)} = P_{\text{crête}} \times \text{PSH} \times \eta$, converti en $\text{mAh/j} = (\text{Wh/j} \div 3,7) \times 1000$, où PSH désigne le nombre d'heures d'ensoleillement équivalent plein soleil (1,5 h/j en hiver, 3,5 h/j en mi-saison, 5,5 h/j en été) et $\eta = 0,60$ le rendement système (régulateur de charge DFR0559 et *derating* réel du panneau). La réserve batterie sans aucun apport solaire est estimée à $4\,000 \text{ mAh} \div 720 \text{ mAh/j}$ soit 5,6 jours.

Scénario (PSH)	Solaire	Bilan	Autonomie
Hiver France (1,5 h/j soleil)	730 mAh/j	+10 mAh/j	Equilibre
Mi-saison (3,5 h/j soleil)	1700 mAh/j	+980 mAh/j	Batterie pleine
Été France (5,5 h/j soleil)	2680 mAh/j	+1960 mAh/j	Batterie Pleine
SOL3W + 4 000 mAh (bilan annuel)	moy. 1700 mAh/j	Rechargé en saison	Autonomie totale

Tableau III. C-1 Bilan énergétique théorique

IV. Beetter Home

IV. A. Conception et architecture matérielle

IV.A.1. Choix des composants

Beetter Home s'articule autour d'un **Raspberry Pi** faisant office de passerelle locale. Le choix du modèle exact n'est pas critique pour le fonctionnement du système : tout Raspberry Pi disposant d'un port UART, d'une interface réseau (Ethernet ou Wi-Fi) et d'une puissance de calcul suffisante pour exécuter Python et Docker peut remplir ce rôle. Le Raspberry Pi 5 a été retenu pour sa capacité à faire tourner sans encombre le récepteur LoRa, l'instance InfluxDB, PostgreSQL, et l'application Flask sans saturation, mais un Raspberry Pi 3B+ ou 4 convient également pour un déploiement à faible nombre de ruches.

Composant	Rôle
Raspberry Pi 5	Unité centrale de traitement. Gère l'acquisition des données, l'analyse et l'affichage.
RFM95 - Grove LoRa 113060006	Module radio Long Range 868MHz. Transmission des données sur plusieurs kilomètres entre Beetter Hive et Beetter Home.

Tableau IV. A-1 Composants matériels du module Beetter Home et leur rôle

IV.A.2. Conception mécanique et boîtier

Le boîtier de Beetter Home a été conçu sous Autodesk Fusion 360 – comme Beetter Hive – et produit par impression 3D (FDM, PLA). La contrainte principale était de loger dans un volume réduit le Raspberry Pi et le module Grove LoRa avec son antenne externe tout en garantissant une ventilation passive suffisante pour le Raspberry Pi en fonctionnement continu.

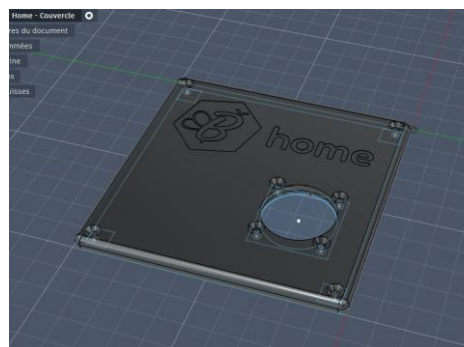
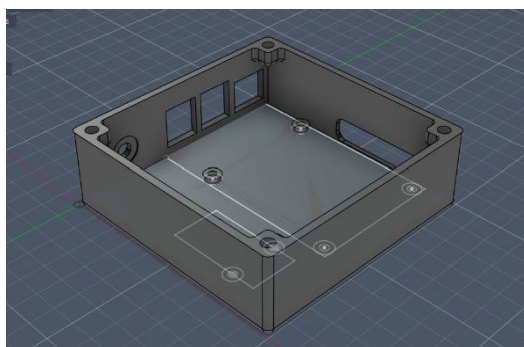


Figure IV. A-1 Design de la boîte (à gauche) et du couvercle (à droite) sur Fusion 360



Le modèle se compose de deux pièces vissables : une boîte avec les plots de fixation aux dimensions exactes du Raspberry Pi 5 et ceux du module LoRa, et un couvercle avec une fixation pour le ventilateur.

IV. B. Architecture logicielle et réseau

L'architecture en quatre couches présentée en II.C constitue le cadre de référence de cette section. On en détaille ici l'implémentation logicielle et réseau au niveau de la passerelle locale.

IV.B.1. Réception et décodage des trames LoRa

Le récepteur (`lora/receiver.py`) décode les blocs ENV et AUD et envoie un relevé JSON à l'API Flask locale (`POST /api/data`). C'est Flask qui écrit ensuite dans InfluxDB, vérifie les seuils et crée les alertes ; le récepteur ne touche plus InfluxDB directement.

IV.B.2. Stockage : InfluxDB et PostgreSQL

Afin de traiter les informations remontées par les ruches (température, humidité, fréquence, et empreintes sonores MFCC), le système repose sur un *pipeline* de données complet : de l'acquisition brute jusqu'à la prédiction comportementale. Cette architecture a été pensée pour être robuste, asynchrone et scalable.

Le projet s'appuie sur une architecture hybride exploitant deux moteurs de bases de données fondamentalement différents, chacun optimisé pour son cas d'usage : **PostgreSQL** et **InfluxDB**.

Avec une séparation des concepts : Données Relationnelles vs Temporelles

- **PostgreSQL (Base Relationnelle)** : Utilisée pour les données « froides » et structurées nécessitant une intégrité stricte.
- **InfluxDB (Base Time-Series)** : Utilisée pour les données « chaudes ». Conçue pour ingérer des milliers de mesures à la seconde, elle est de type *Schema-less* (sans schéma figé).

Pour optimiser les performances de lecture (*dashboard*) et protéger la carte mémoire du Raspberry Pi, les données temporelles d'InfluxDB sont séparées en deux « *Buckets* » (seaux) distincts (*Pipeline ETL*) :

1. **Le Bucket 1 (sensors)** : C'est le réceptacle des données brutes. Il ingère les températures et les tableaux complexes de MFCC. Une politique de rétention courte (ex: 30 jours) y est appliquée pour éviter la saturation du disque.
2. **Le Bucket 2 (predictions)** : C'est le *bucket* d'analyse. Il ne reçoit aucune donnée de l'ESP32. Il est alimenté exclusivement par notre intelligence artificielle qui y dépose une probabilité pour chaque danger (ex : « 0,85 »). Ces données numériques, très légères, peuvent être conservées indéfiniment pour créer un historique inter-saisonnier.



IV. C. Architecture et organisation du code du tableau de bord web

Le tableau de bord web (dashboard) de Beetter Home est une application web servie localement par un serveur Flask Python, accessible depuis n'importe quel navigateur sur le réseau local. Il constitue l'interface principale de supervision pour l'apiculteur présent sur site, offrant une vue complète et en temps réel de l'état de sa flotte de ruches.

IV.C.1. Organisation en *blueprints* Flask

Flask permet de structurer une application web en modules indépendants appelés *blueprints*. Chaque *blueprint* regroupe les routes HTTP, la logique métier et les templates HTML associés à une section fonctionnelle de l'application. Cette organisation garantit une séparation claire des responsabilités et facilite la maintenance indépendante de chaque module.

Le tableau de bord est divisé en quatre *blueprints* principaux :

Blueprint	Routes principales	Responsabilité
dashboard	/, /overview	Vue d'ensemble de la flotte, synthèse des statuts, carte interactive et croisement météo
beehives	/beehives/<id>, /beehives/<id>/data	Détail par ruche, graphes de mesures avec fenêtres temporelles navigables, export CSV
alerts	/alerts, /alerts/history	Page d'alertes actives et historique, filtrage par ruche, acquittement manuel
calendar	/calendar	Calendrier des interventions apicoles, création et suivi des événements par ruche

Tableau IV. C-1 Blueprints Flask du tableau de bord web

IV.C.2. Moteur de templates Jinja2

Les pages HTML sont générées côté serveur grâce au moteur de templates Jinja2, intégré nativement à Flask. Jinja2 permet d'injecter dynamiquement les données issues d'InfluxDB et de PostgreSQL directement dans les pages HTML avant leur envoi au navigateur. Ce fonctionnement dit de rendu côté serveur (Server-Side Rendering) garantit que l'apiculteur reçoit toujours une page à jour, même sans JavaScript activé.

Les templates partagent une structure commune basée sur l'héritage Jinja2 : un template de base (base.html) définit la mise en page générale (*header*, sidebar, footer) et chaque page hérite de ce template en ne redéfinissant que son contenu spécifique. Cela évite toute duplication de code HTML entre les différentes vues.



IV.C.3. Sources de données

Le tableau de bord interroge deux bases de données distinctes selon la nature de l'information demandée :

- **InfluxDB** reçoit toutes les mesures temporelles des capteurs : températures intérieure et extérieure, humidité, fréquence acoustique, amplitude RMS et coefficients MFCC. Ces données sont requêtées via le langage Flux natif d'InfluxDB, qui permet d'effectuer des agrégations, des rééchantillonnages et des filtrages temporels directement au niveau de la base.
- **PostgreSQL** stocke les données structurales et relationnelles : identités des ruches, comptes utilisateurs, seuils d'alerte configurés, événements du calendrier et journal des alertes. Ces données sont accédées via SQLAlchemy, l'ORM Python intégré à Flask.

IV.C.4. Librairies front-end

Librairie	Rôle dans le tableau de bord
Bootstrap 5	Framework CSS responsive gérant la mise en page, les composants UI (cartes, badges, modales) et l'adaptation mobile
Chart.js	Génération des graphes interactifs de séries temporelles avec zoom, fenêtres temporelles et légendes dynamiques
Leaflet	Carte interactive OpenStreetMap affichant la position géolocalisée de chaque ruche avec son statut en temps réel
FullCalendar	Calendrier des interventions apicoles avec vue mensuelle, hebdomadaire et gestion des événements par ruche
Fetch API (JS)	Requêtes AJAX vers les <i>endpoints</i> Flask pour rafraîchir les données sans rechargement de page (statuts, mesures)

Tableau IV. C-2 Librairies front-end du tableau de bord

IV.C.5. Tâches automatisées avec APScheduler

APScheduler est une librairie Python permettant d'exécuter des tâches planifiées en arrière-plan, au sein même du processus Flask. Trois tâches tournent en continu sur Beetter Home :

Tâche	Fréquence	Description
Détection perte de signal	Toutes les 5 min	Vérifie si chaque ruche a envoyé des données dans la fenêtre attendue. Génère une alerte 'Sans données' si le délai est dépassé.



Tâche	Fréquence	Description
Push vers le serveur distant	Toutes les 30 s	Envoie les nouvelles trames reçues depuis Beetter Hive vers Beetter Connect via POST /api/push.
Résumés journaliers	Chaque nuit à 00h00	Calcule et stocke un résumé des mesures de la journée (min, max, moyenne) par ruche dans PostgreSQL.

Tableau IV. C-3 Tâches automatisées APScheduler sur Beetter Home

Ces tâches s'exécutent indépendamment des requêtes HTTP : même si personne ne consulte le tableau de bord, la détection de perte de signal et le push vers le serveur distant continuent de fonctionner en arrière-plan.

IV. D. Fonctionnalités du *dashboard*

IV.D.1. Surveillance environnementale continue

Le système assure une mesure ininterrompue des paramètres environnementaux critiques à la santé des colonies :

- Température intérieure et extérieure de la ruche en temps réel ;
- Taux d'humidité, indicateur direct de l'état sanitaire de la colonie ;
- Son et Fréquence de la ruche, indice du comportement des abeilles ;
- Niveau de lumière extérieure, permettant de détecter une activité nocturne, ou une exposition directe au soleil.

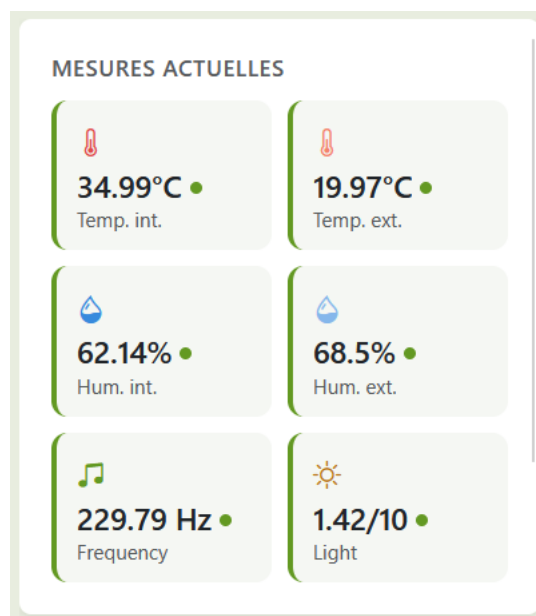


Figure IV. D-1 Exemple d'interface, affichant les données de la ruche

Chaque mesure environnementale est comparée à une plage de référence à trois zones (normale, avertissement, critique). Une valeur sortant de la zone normale sans atteindre le seuil critique fait basculer la carte de la ruche concernée en état *Agité*, et le dépassement du seuil critique déclenche l'état *Critique* ainsi que la création automatique d'une alerte horodatée dans le journal. Les seuils sont définis par mesure (température, humidité, fréquence sonore, luminosité) et peuvent être ajustés sans modification du code.

Valeur	Type	Plages		
		Normale	Risque	Critique
Température	Intérieur	32 – 38 °C	28 – 42 °C	< 28 ou > 42 °C
	Extérieur	5 – 35 °C	0 – 40 °C	< 0 ou > 40 °C
Humidité	Intérieur	50 – 75 %	40 – 85 %	< 40 ou > 85 %
	Extérieur	30 – 90 %	20 – 95 %	< 20 ou > 95 %
Fréquence Sonore	Intérieur	180 – 320 Hz	150 – 400 Hz	< 150 ou > 400 Hz

Valeur	Type	Plages		
		Normale	Risque	Critique
	Extérieur	150 – 400 Hz	150 – 500 Hz	< 150 ou > 500 Hz
Lumière	Extérieur	0 – 5	0 – 10	–

Tableau IV. D-1 Seuils arbitraires (à ajuster)

Le changement d'état est piloté automatiquement par les seuils : une ruche revient d'elle-même en zone *Calme* dès que toutes ses valeurs repassent dans la plage normale, même si un administrateur avait fixé manuellement un autre statut. Cette règle évite qu'un changement manuel ne masque durablement une situation résolue. Les états *Essaimage* et *Prédateur* sont aujourd'hui déclenchés par le *pipeline* de classification IA (voir IV.E), qui écrira directement ces états "observés" sans passer par une saisie humaine.

IV.D.2. Gestion de flotte et interface utilisateur

La plateforme web centralise l'administration de l'ensemble des ruches depuis une Interface Homme-Machine (IHM) unique.

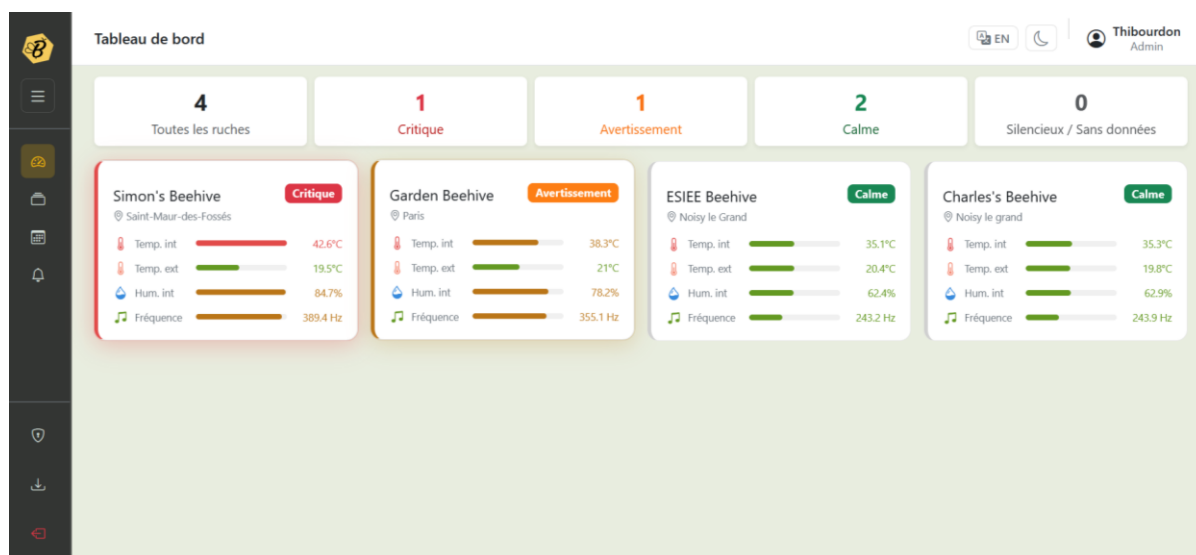


Figure IV. D-2 Tableau de bord temps réel affichant l'état synthétique de chaque ruche

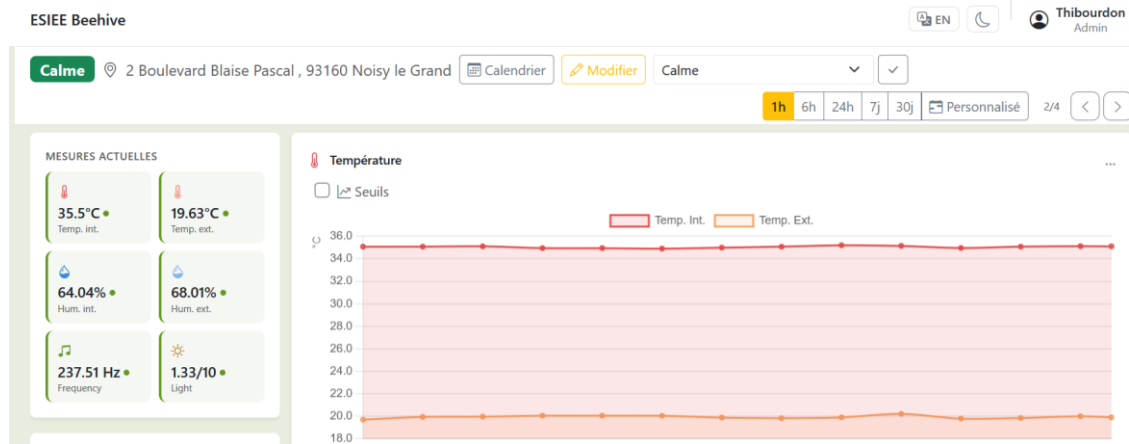


Figure IV. D-3 Historique des données navigable avec sélection de plages temporelles personnalisées

Ruches

Rechercher une ruche...

Toutes Critique Alerte Calme Hors ligne

Vue technique Rafralchir + Ajouter une ruche

NOM	EMPLACEMENT	ÉTAT	CONNEXION	EUI APPAREIL	SF / BW	FRÉQ. (MHZ)	ACTIONS
Garden Beehive Thibourdon	12 rue de la Paix, 75000 Paris	Avertissement	En ligne	-	SF7 / 125kHz	868.0	...
ESIEE Beehive Thibourdon	2 Boulevard Blaise Pascal, 93160 Noisy le Grand	Calme	En ligne	-	SF7 / 125kHz	868.0	...
Simon's Beehive Thibourdon	40 avenue Foch, 94100 Saint-Maur-des-Fossés	Critique	En ligne	-	SF7 / 125kHz	868.0	...
Charles's Beehive Thibourdon	2 Boulevard Blaise Pascal, 91260 Noisy le grand	Calme	En ligne	-	SF7 / 125kHz	868.0	...

Figure IV. D-4 Gestions globales des ruches




Beetter

Welcome — first-time setup

Step 1 of 2
Create your admin account

Username

Email

Password

Confirm password

Create account & get started


Beetter

Welcome — first-time setup

Step 2 of 2
Add your first hive

Welcome to Beetter! Your admin account has been created.

Hive ID

4 letters/digits. Must match the `beehive_id` your sensor sends.

Hive name

City (optional)

Create hive & finish

[Skip for now →](#)

Figure IV. D-5 Page de configuration

Une page Préférences centralise les réglages propres à chaque utilisateur : langue de l'interface, format d'heure (12h/24h), premier jour de la semaine, unité de température, et options d'accessibilité (contraste élevé, texte agrandi, réduction des animations). Ces réglages sont conservés en base et appliqués indépendamment pour chaque compte, ce qui permet à plusieurs apiculteurs de partager un même parc de ruches sans imposer leurs préférences d'affichage aux autres utilisateurs.

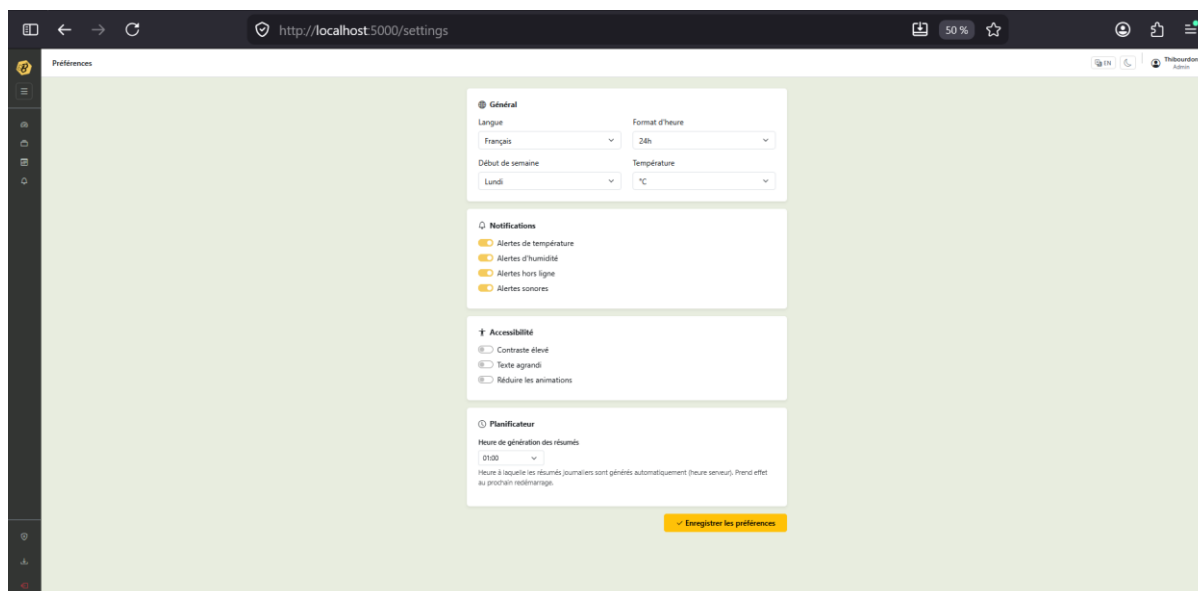


Figure IV. D-6 Page des préférences utilisateur

À ce jour, la gestion de flotte s'organise autour de la ruche individuelle. Un regroupement par “rucher” (ensemble de ruches partageant un même emplacement géographique), envisagé pour faciliter la

lecture du *dashboard* à mesure que le nombre de ruches suivies augmente, a été maqueté et discuté en fin de période mais reste à implémenter ; il constitue une perspective d'évolution proche du module de gestion de flotte (voir V.D).

IV.D.3. Alertes, export et API REST

Chaque changement d'état d'une ruche (automatique, par franchissement de seuil, ou manuel) génère une notification alerte ainsi qu'une entrée dans un journal d'alertes horodaté, conservant l'ancien statut, le nouveau statut et la source du changement (seuil, prédiction IA et changement manuel). Ce journal permet à l'apiculteur de reconstituer l'historique complet d'une ruche sans avoir à corrélérer manuellement les courbes de mesures.

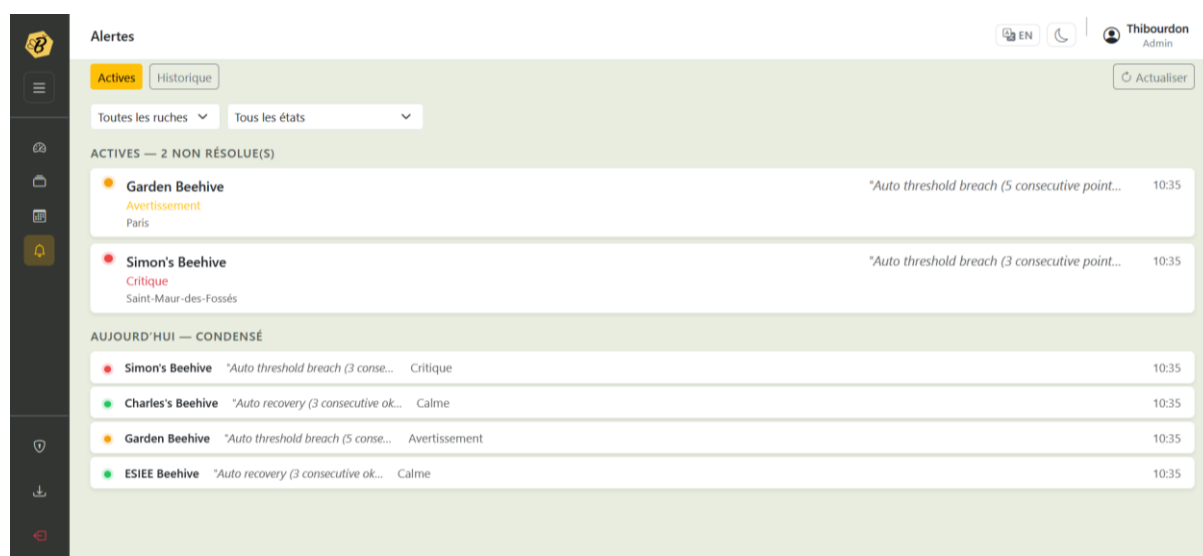


Figure IV. D-7 Page des alertes

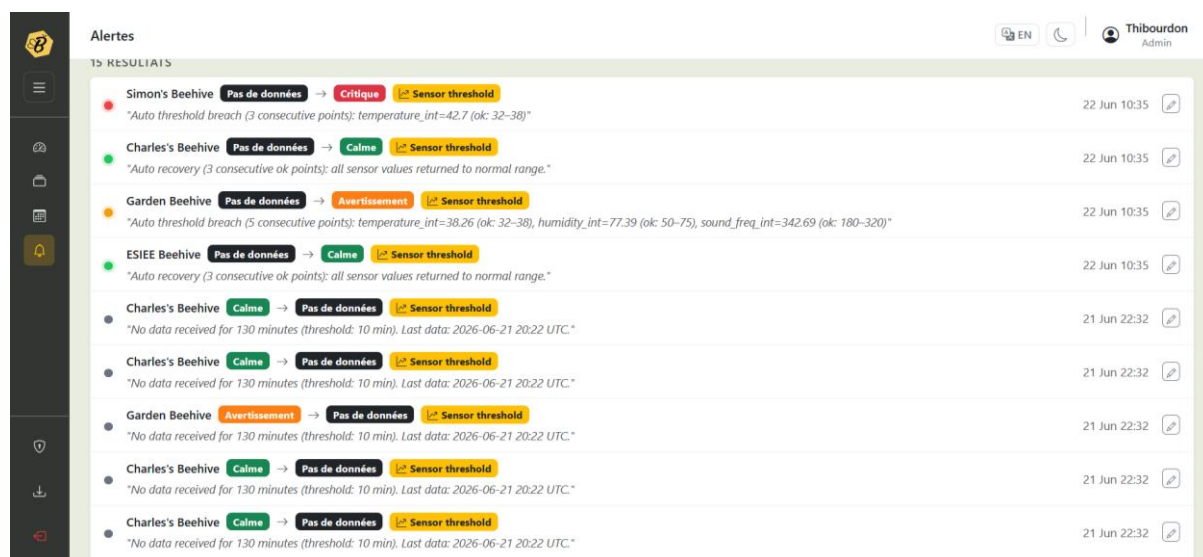


Figure IV. D-8 Filtre d'alertes

L'export des données s'effectue au format CSV depuis une page dédiée : l'utilisateur sélectionne une ou plusieurs ruches, une période, puis les champs à inclure parmi les mesures scalaires et, séparément, les 26 coefficients MFCC intérieurs et extérieurs. Cette séparation reflète l'usage : les mesures scalaires servent au suivi courant, les MFCC alimentent principalement le *pipeline* d'apprentissage automatique (voir IV.E).

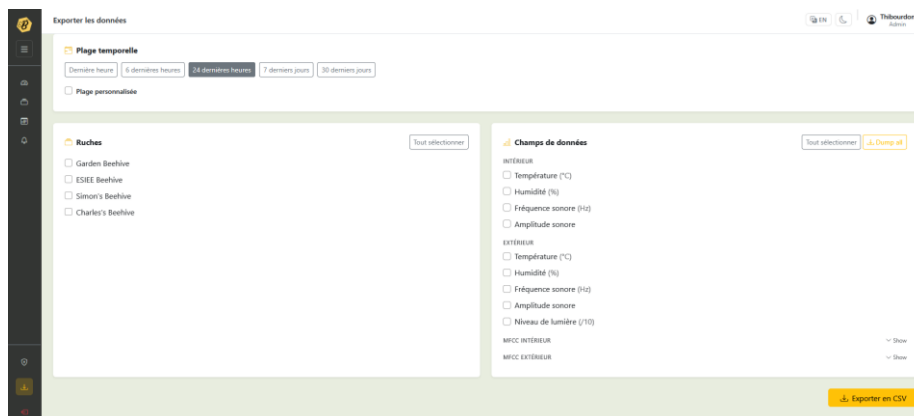


Figure IV. D-9 Page d'export des données

Au-delà de l'export ponctuel, un résumé quotidien (température, humidité et fréquence sonore moyennes, nombre d'alertes, statut de fin de journée) est calculé automatiquement pour chaque ruche à une heure paramétrable, puis affiché directement sur la page de détail de la ruche et intégré au calendrier. L'apiculteur dispose ainsi d'une vue de tendance journalière sans devoir interroger l'historique brut des mesures.

Un calendrier partagé permet de planifier et de tracer les interventions sur le rucher : inspections, traitements, récoltes, essaimages et événements divers, chacun pouvant être rattaché à une ruche précise ou laissé global. Combiné aux résumés quotidiens automatiques, le calendrier devient à la fois un outil de planification (anticiper la prochaine inspection) et un journal de bord rétrospectif de l'activité du rucher.

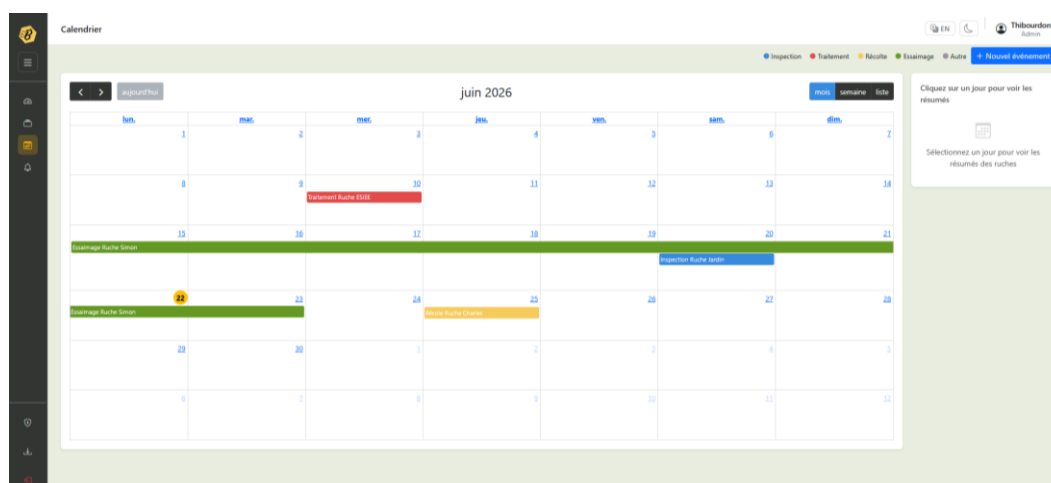


Figure IV. D-10 Page calendrier



IV. E. Intelligence artificielle et classification comportementale

IV.E.1. Problématique et états classifiés

Le système intègre un modèle d'intelligence artificielle capable de reconnaître l'état comportemental d'une colonie à partir des données acoustiques et environnementales transmises par Beetter Hive. Ce modèle repose sur une approche dite contrastive : plutôt que d'apprendre à partir d'exemples préalablement étiquetés, il apprend par comparaison, en identifiant ce qui distingue une situation normale d'une situation anormale au fil des données reçues. Concrètement, il s'appuie sur deux analyseurs indépendants - l'un dédié aux capteurs intérieurs, l'autre aux capteurs extérieurs – qui transforment chaque mesure en une représentation numérique synthétique. En comparant en permanence ces deux représentations, le modèle détecte les ruptures de corrélation entre l'intérieur et l'extérieur de la ruche, caractéristiques d'un événement particulier. Une fois suffisamment de données collectées et confirmées par l'apiculteur, il devient capable de classifier automatiquement l'état de la colonie parmi six situations cibles : normal, pré-essaimage, essaimage, compétition royale, orphelinage et attaque. Ces états sont stockés sous forme de probabilités dans InfluxDB qui sont ensuite récupérées par le tableau de bord pour un affichage en temps réel pour l'apiculteur.

Événement détecté	Description
Essaimage imminent	Détection anticipée grâce à la signature acoustique caractéristique précédant l'événement. Permet à l'apiculteur d'intervenir avant la perte d'un essaim.
Stress de la colonie	Identification d'anomalies comportementales : présence d'un intrus, chaleur excessive, manque de ressources.
Perte de reine	Détection de l'orphelinage par analyse du profil sonore de la colonie en l'absence de reine.
Intrusion de prédateurs	Reconnaissance acoustique des frelons asiatiques (<i>Vespa velutina</i>) et autres prédateurs.
Classification biologique	Distinction bourdons / abeilles ouvrières par analyse comparative de leurs fréquences de vol ^[4] .

Tableau IV. E-1 Événements comportementaux détectables par analyse acoustique (Annexe VII. C. Acoustique Apicole)

IV.E.2. Analyse acoustique prédictive

L'apiculture acoustique^{[4][5]} constitue le principal levier différenciant de Beetter. L'analyse spectrale (par FFT et MFCC) des bourdonnements permet d'identifier des événements comportementaux impossibles à détecter par les seuls capteurs environnementaux :



Les événements détectés s'inscrivent dans des délais caractéristiques : l'essaimage peut être anticipé 24 à 72 heures avant le départ de l'essaim, le chant de la reine vierge (*tooting*) est détectable 12 à 24 heures avant l'éclosion^{[8][9]}, et le stress d'une colonie soumise à un prédateur (frelon asiatique) est perceptible en quelques minutes. Ces fenêtres temporelles déterminent directement la valeur opérationnelle des alertes pour l'apiculteur (Annexe VII. C. et Annexe VII. D.).

IV.E.3. Données d'entrée et vecteur de caractéristiques

Chaque cycle de mesure produit, pour chacun des deux capteurs (intérieur et extérieur), un vecteur de 17 grandeurs : la température, le taux d'humidité, la fréquence dominante du bourdonnement, l'amplitude RMS, et les 13 coefficients MFCC calculés par l'ESP32 (indices 0 à 12). Ces paramètres sont ensuite normalisés statistiquement - ramenés à une moyenne nulle et un écart-type unitaire - de sorte que l'échelle absolue des grandeurs n'influence pas l'apprentissage. Les statistiques de normalisation sont calculées une seule fois lors d'une phase de calibration correspondant aux premières semaines de déploiement, puis figées pour garantir la cohérence des représentations tout au long de la vie du système.

Les données d'entraînement sont extraites d'InfluxDB en interrogeant le *Bucket* sensors. Les lignes dont les coefficients MFCC sont absents peuvent être conservées avec la valeur zéro afin d'alimenter un pré-entraînement partiel avant la mise en service complète du Firmware.

IV.E.4. Architecture du modèle contrastif

Le modèle repose sur quatre composants distincts, chacun ayant un rôle précis dans la chaîne de traitement.

Le modèle s'appuie sur deux encodeurs indépendants : l'un dédié au capteur intérieur, l'autre au capteur extérieur. Chacun est un réseau de neurones à deux couches denses qui compresse progressivement le vecteur de 17 dimensions en une représentation compacte de 32 dimensions, avec normalisation et activation non-linéaire entre les couches. Les deux encodeurs partagent la même architecture mais pas les mêmes paramètres internes : les capteurs intérieur et extérieur évoluent dans des environnements physiquement distincts, ce qui justifie que chacun développe sa propre spécialisation plutôt qu'une représentation commune imposée.

Une tête de projection intervient exclusivement lors de la phase de pré-entraînement : chaque sortie d'encodeur y est projetée vers un espace de dimension réduite, normalisé sur la sphère unité. Cette contrainte géométrique garantit que la distance entre deux représentations est purement directionnelle, ce qui stabilise les comparaisons et rend l'apprentissage contrastif plus efficace. Cette composante est ensuite écartée lors de l'inférence en production.

Une tête de classification prend le relais lors de la phase d'affinage supervisé et à l'inférence : les sorties des deux encodeurs ($32 + 32 = 64$ dimensions au total) sont concaténées et soumises à un classificateur produisant six scores – un par état comportemental cible. La concaténation plutôt que la simple moyenne préserve l'asymétrie entre les deux capteurs : de nombreux états ne sont identifiables que par la différence entre l'intérieur et l'extérieur. Un essaimage, par



exemple, se traduit par une activité extérieure anormalement élevée couplée à une activité intérieure normale – une signature que la moyenne des deux flux effacerait.

Chaque encodeur (BeehiveEncoder) est composé de deux couches denses successives : la première projette le vecteur de 17 dimensions vers 64 neurones, la seconde réduit ces 64 neurones à 32. Entre chaque couche, une normalisation par lot (BatchNorm) stabilise les activations avant la non-linéarité (ReLU), ce qui s'avère particulièrement important pour les jeux de données tabulaires de taille réduite. Avec deux encodeurs indépendants, deux têtes de projection et la tête de classification, le modèle totalise environ 9 100 paramètres – une taille volontairement réduite permettant la convergence avec quelques centaines à quelques milliers d'exemples, sans nécessiter d'unité de calcul spécialisée.

Composants	Entrée	Sortie	Paramètres
BeehiveEncoder intérieur	17	32	2912
BeehiveEncoder extérieur	17	32	2912
Tête de projection (×2)	32	16	1024
Tête de classification	64	6	2278
Total			9126

Tableau IV. E-2 Composants formant le modèle d'IA contrastif avec leurs paramètres respectifs

IV.E.5. Deux phases d'entraînement

La première phase, dite de pré-entraînement, opère en l'absence de données étiquetées lors des premières semaines de déploiement, en exploitant une propriété intrinsèque du système : à un instant donné, les deux capteurs observent simultanément le même état de la ruche. L'apprentissage consiste alors à rapprocher les représentations du capteur intérieur et du capteur extérieur issues d'un même instant (paire positive, car la ruche a le même état), tout en éloignant celles d'instantanés différents (paire négative, pas forcément le même état). Ce signal d'apprentissage – la simultanéité des deux flux – permet au modèle de structurer son espace de représentation autour des états comportementaux réels, avant même que le moindre événement ait été étiqueté. Cette phase peut être lancée dès que quelques centaines d'observations brutes sont disponibles et produit un graphique sur lequel apparaissent progressivement des nuages de points (*clustering*), révélateurs des différents états comportementaux.

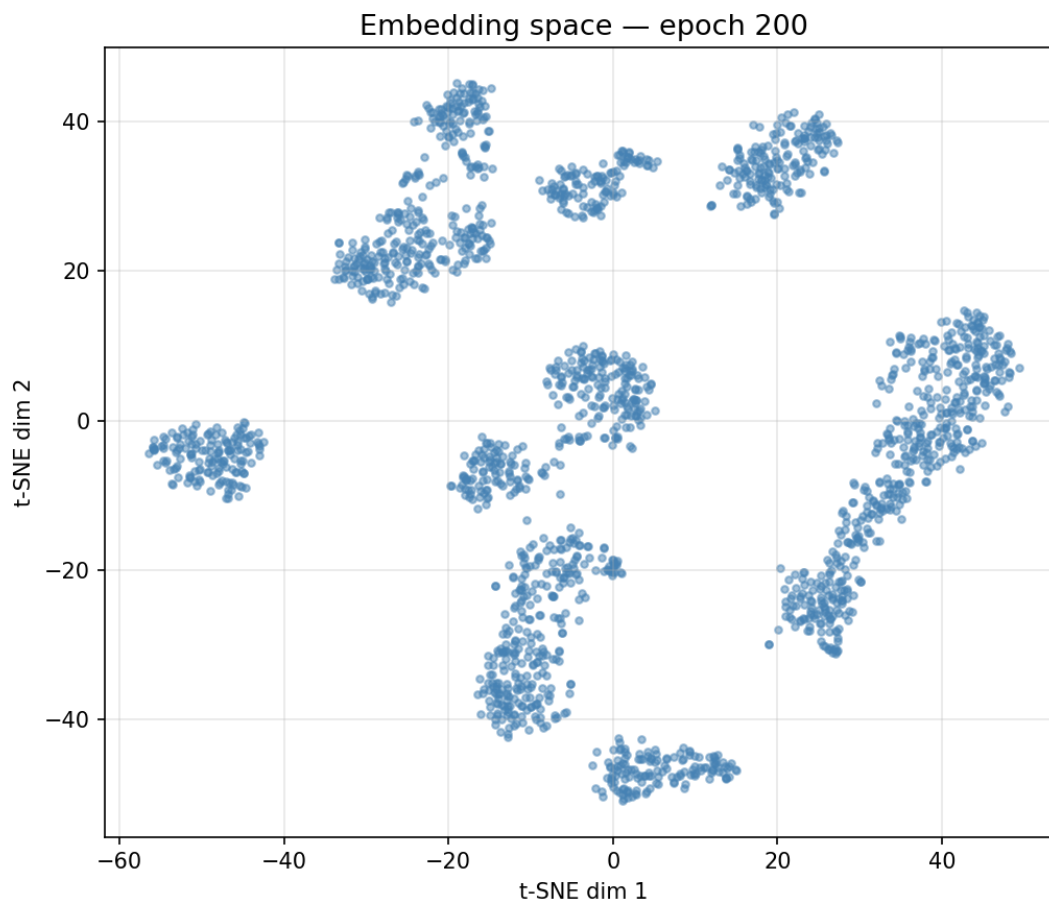


Figure IV. E-1 Graphique t-SNE représentant l'apprentissage du modèle après la phase 1

La deuxième phase, dite d'affinage supervisé, s'enclenche lorsque l'apiculteur a observé et annoté des événements particuliers : ces étiquettes permettent d'associer définitivement chaque région de l'espace de représentation à l'un des six états cibles. Pour accélérer la labellisation – qui constitue le principal obstacle pratique – un mécanisme d'auto-étiquetage exploite les trois modèles *Random forest* développés en parallèle sur des enregistrements audio de référence. Ces modèles proposent automatiquement des étiquettes pour les états « attaque de frelon », « essaimage » et « chant de la reine » sur les données historiques, que l'apiculteur valide ou corrige avant de lancer l'affinage. Les deux approches sont ainsi complémentaires : les *Random forest* fournissent les étiquettes de départ, tandis que le modèle contrastif généralise la classification à l'ensemble des six états.

IV.E.6. Déroulement de l'entraînement

L'entraînement se déroule en deux temps distincts. Lors de la Phase 1, le modèle est entraîné sur les données brutes non étiquetées pendant 200 cycles complets (*epochs*), avec un taux d'apprentissage décroissant selon un planning cosinusoidal. La progression peut être visualisée grâce à des projections *t-SNE* générées périodiquement : un nuage de points aléatoire en début d'entraînement qui se structure progressivement en groupes distincts signale que le modèle commence à différencier les états comportementaux. Lors de la Phase 2, le *backbone* encodeur est gelé – ses paramètres ne sont plus modifiés – et seule la tête de classification est entraînée



pendant 100 cycles supplémentaires sur les données étiquetées. Cette stratégie de gel prévient le surapprentissage en limitant à environ 2 300 les paramètres effectivement mis à jour sur un jeu de données étiquetées nécessairement restreint.

IV.E.7. Complémentarité *Random forest* / modèle contrastif

L'objectif de l'IA est de classifier l'état de la colonie (Normal, Essaimage, Attaque de frelon, Chant de la reine) à partir de la signature acoustique interne et externe de la ruche.

Les trois modèles *Random forest* et le modèle contrastif ont des rôles distincts et non concurrents dans l'architecture finale. Les *Random forest* constituent un premier détecteur rapide, entraîné sur des enregistrements audios de référence téléchargés depuis des bases de données publiques, opérant en mode binaire (événement présent ou absent) sur les coefficients MFCC d'un seul capteur à la fois. Le modèle contrastif, quant à lui, est entraîné directement sur les données LoRa réelles de la ruche surveillée, couvre les six états comportementaux simultanément, et tire parti de la corrélation entre les deux capteurs – une information absente des *Random forest* qui n'exploitent qu'un seul flux. Dans la pratique, les sorties des *Random forest* servent d'amorce à la labellisation des données historiques, et le modèle contrastif prend ensuite le relais pour la classification en conditions réelles. Ce volet reste en phase de prototypage : la chaîne complète (acquisition, encodeurs, classification, écriture des prédictions) est implémentée et fonctionne de bout en bout, mais l'affinage supervisé et l'évaluation quantitative sont actuellement limités par le faible volume de données étiquetées issues du déploiement initial.

IV.E.8. Intégration dans le *pipeline* et logique d'alertes

À l'inférence, chaque nouvelle trame LoRa reçue est normalisée, soumise aux deux encodeurs, et le classificateur produit six probabilités correspondant aux six états comportementaux. Ces probabilités sont stockées dans un espace dédié d'InfluxDB (*Bucket predictions*), séparé des données brutes de capteurs, afin de ne pas polluer les historiques de mesures et de permettre une conservation longue durée pour l'analyse inter-saisonnière. Le tableau de bord récupère ces probabilités pour afficher l'état courant de chaque ruche et déclencher les alertes appropriées. La logique d'alerte s'appuie sur une fenêtre glissante : un état n'est confirmé que si sa probabilité dépasse 40 % sur trois lectures consécutives, évitant les fausses alertes dues à des trames isolées bruitées.

IV.E.9. *Feature Engineering* et Extraction

Le son brut n'est pas transmis au serveur (trop lourd pour le réseau LoRa). C'est le microcontrôleur (ESP32) qui calcule localement **13 coefficients MFCC** (Mel-Frequency Cepstral Coefficients). Contrairement à une simple mesure de volume en décibels, les MFCC agissent comme une « empreinte digitale » ou un prisme, capturant la texture et le timbre exact du bourdonnement. Ces 13 variables constituent le vecteur d'entrée (*Features*) pour les modèles :



- Le coefficient **0** correspond à l'**Énergie Globale**. C'est le seul facilement interprétable pour un humain. Il représente le « volume » global du son.
- Les coefficients **1 à 4** correspondent à la **Forme globale (Les Formants)**. Ils décrivent la structure générale du son. Par exemple s'il s'agit d'un son grave, sourd, ou plutôt aigu. Permet de distinguer les différentes composantes d'un son.
- Les coefficients **5 à 12** correspondent aux **Détails fins**. Ils capturent les micro-variations du timbre (les harmoniques par exemple). C'est ce qui permet de faire la différence entre deux sons très proches.

Ainsi, à partir de la transformée de Fourier :

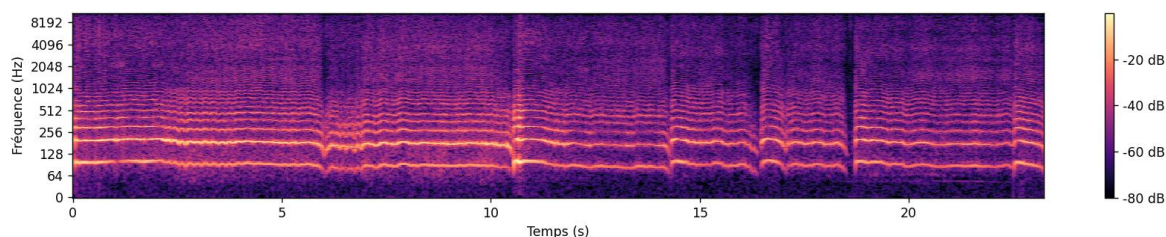


Figure IV. E-2 Spectrogramme du bourdonnement d'un frelon asiatique (FFT)

On en déduit les 13 coefficients MFCC :

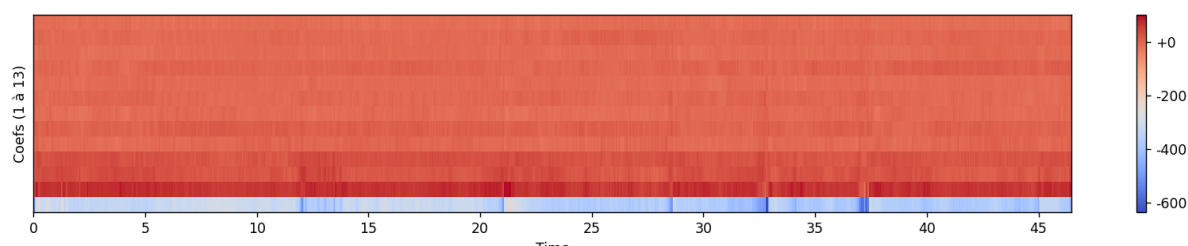


Figure IV. E-3 Empreinte MFCC du bourdonnement d'un frelon asiatique

IV.E.10. Entraînement et Data Augmentation

Trois modèles spécialisés ont été développés basés sur l'algorithme des **Forêts Aléatoires (Random forest Classifieur)** via la bibliothèque Scikit-Learn. Pour pallier le manque de données réelles d'attaques ou d'essaimage, une stratégie rigoureuse de *Data Augmentation* a été appliquée sur nos fichiers audio d'entraînement :

- **Ajout de bruit de fond** : Création de clones virtuels avec injection de bruit blanc (simulation du vent).
- **Mixage dynamique** : Superposition aléatoire de sons de frelons ou de reines sur des enregistrements de ruches saines, à des volumes variables.
- **Séparation stricte (anti-fuite de données)** : Le jeu de données est scindé en données d'entraînement (*Train*) et de test (*Test*) avant toute augmentation, garantissant que le modèle est évalué sur des situations purement réelles qu'il n'a jamais rencontrées.



IV.E.11. Sérialisation des Modèles (Mise en Production)

Une fois les modèles entraînés avec une précision satisfaisante, ils sont sérialisés et sauvegardés sur le disque dur à l'aide de la bibliothèque **Joblib** (fichiers .pkl). Cette étape fige les « cerveaux » de l'IA, permettant au système de redémarrer instantanément en production sans nécessiter de nouveaux calculs d'entraînement économisant de l'énergie, du temps et de la puissance de calcul.

IV.E.12. Le *Watchdog* d'Inférence 24/7 (Temps Réel)

Le système prend vie grâce à un script Python indépendant fonctionnant en tâche de fond (*Watchdog*) :

1. **Lecture (*Pull*)** : Toutes les 5 minutes, le script interroge le *Bucket 1* d'InfluxDB pour récupérer les 13 derniers MFCC.
2. **Prédiction (*Predict*)** : Il soumet ce vecteur aux trois modèles *Random forest* chargés en mémoire. Au lieu d'une décision binaire brute, il utilise la fonction `predict_proba` pour obtenir un pourcentage de certitude (ex : probabilité d'attaque).
3. **Décision et Écriture (*Push*)** : Si une probabilité dépasse un seuil de confiance critique (défini à 75 %), le script qualifie l'événement (ex: « attaque_frelon ») et écrit immédiatement cette conclusion dans le *Bucket 2* (`data_ia`).

C'est cette donnée finale, propre et interprétée, qui est ensuite consommée par le serveur web (Flask) pour afficher des alertes compréhensibles sur le *smartphone* de l'apiculteur.

En synthèse, les fonctionnalités développées couvrent l'intégralité du cycle de surveillance : de la captation physique des données à leur restitution à l'apiculteur sous forme d'alertes intelligentes et de visualisations exploitables.



V. Beetter Connect et application mobile Beetter

Beetter Connect constitue la troisième et dernière couche de l'architecture Beetter, après la captation embarquée (Beetter Hive) et l'agrégation locale (Beetter Home). Son rôle est de centraliser à distance les données provenant de plusieurs Beetter Home, de les stocker de façon durable, et de les exposer via une API REST sécurisée. Il constitue le point d'entrée unique de l'application mobile Beetter, mais aussi un point d'intégration potentiel pour des services tiers souhaitant accéder aux données apicoles. C'est la brique qui permet de passer d'une solution mono-rucher en réseau local à une plateforme multi-ruchers accessible depuis n'importe quel terminal connecté.

V. A. Architecture du serveur distant

La séparation entre la passerelle locale (Beetter Home) et le serveur distant (Beetter Connect) répond à un principe fondamental d'architecture distribuée : ne pas faire supporter à un composant edge des responsabilités cloud. Beetter Home est déployé sur un Raspberry Pi avec des ressources limitées, exposé à des conditions environnementales difficiles et sans garantie de connexion internet permanente. Lui confier la gestion multi-utilisateurs, l'exposition publique et la rétention long terme des données serait à la fois risqué en termes de sécurité et inadapté à ses contraintes matérielles.

Beetter Connect est conçu pour être scalable : une seule instance peut agréger les données de plusieurs Beetter Home dispersés géographiquement, chacun desservant un groupe rucher différent. L'apiculteur dispose ainsi d'une vue unifiée de l'ensemble de sa flotte depuis un point d'accès unique.

V.A.1. Stack logicielle

Beetter Connect repose sur la même *stack* que Beetter Home, pour des raisons de cohérence et de maintenabilité :

Composant	Rôle
Flask Python	Framework web assurant le routage des requêtes HTTP, la logique métier et l'exposition de l'API REST
InfluxDB	Base de données temporelles recevant les mesures capteurs (org : beetter_srv, bucket : sensors)
PostgreSQL	Base relationnelle gérant les entités structurales : comptes utilisateurs, configuration des ruches, journal des alertes
Docker compose	Orchestration des conteneurs Flask, InfluxDB et PostgreSQL en environnement isolé et reproductible

Tableau V. A-1 Stack logicielle de Beetter Connect

L'application est conteneurisée via Docker Compose, ce qui garantit une installation identique quel que soit l'environnement hôte et simplifie le déploiement sur un VPS cloud.



V.A.2. Déploiement actuel et perspective cloud

Dans la version actuelle du projet, Beetter Connect est déployé sur une machine virtuelle Debian hébergée en local, accessible sur le port 5001. Cette configuration a été choisie pour valider la chaîne fonctionnelle complète — de la captation jusqu'à l'affichage mobile — sans dépendance à un environnement cloud externe. Elle implique cependant que l'application mobile et le serveur doivent être sur le même réseau Wi-Fi, ce qui limite l'usage à une présence physique sur le site du rucher.

Le passage en production sur un VPS cloud est l'étape suivante identifiée. Une solution comme OVH Cloud permettrait d'exposer Beetter Connect via une adresse IP publique fixe, avec activation du protocole HTTPS (certificat Let's Encrypt) pour sécuriser les échanges. Ce déploiement rendrait la plateforme accessible depuis n'importe où dans le monde, et permettrait à plusieurs Beetter Home de s'y connecter simultanément. Aucune modification de l'architecture applicative n'est nécessaire pour ce passage à l'échelle : le code gère déjà les données multi-sources via l'identifiant `hive_id` associé à chaque trame.

V. B. API REST et sécurité

L'API REST de Beetter Connect est le seul point de contact entre le serveur et ses clients : Beetter Home d'un côté pour pousser les données, l'application mobile de l'autre pour les consulter. Cette conception respecte le principe de séparation des couches : les clients ne connaissent pas la structure interne de la base de données, et le serveur ne dépend d'aucune implémentation client spécifique. Chaque ressource est identifiée par une URL, chaque opération par un verbe HTTP standard (GET, POST, PUT, DELETE), et les réponses sont formatées en JSON. Cette standardisation facilite l'intégration future de clients tiers — un tableau de bord web distant, une application desktop, ou une intégration domotique.

V.B.1. Authentification par JWT

L'accès à l'API est protégé par JSON Web *Token* (JWT). Le mécanisme fonctionne en trois étapes :

- L'utilisateur s'authentifie via `/api/auth/login` avec ses identifiants. Le serveur vérifie les informations en base PostgreSQL et retourne un *token* JWT signé avec une clé secrète.
- L'application cliente stocke ce *token* et l'inclut dans toutes les requêtes suivantes via le *header* HTTP : `Authorization: Bearer <token>`.
- Le serveur valide la signature du *token* à chaque requête sans consultation de la base. Ce fonctionnement stateless réduit la charge et facilite la scalabilité horizontale.

Les mots de passe sont stockés hachés dans PostgreSQL via `bcrypt`. Aucun mot de passe en clair n'est conservé ni transmis après la phase d'authentification initiale.



V.B.2. Authentification machine-à-machine pour Beetter Home

Le mécanisme JWT est conçu pour les interactions avec un utilisateur humain : il suppose une phase d'identification interactive avec saisie d'identifiants. Or Beetter Home est un service autonome qui doit s'authentifier automatiquement, sans intervention humaine et potentiellement à haute fréquence (toutes les 30 secondes). L'authentification repose donc sur une clé API statique, générée une seule fois lors de la configuration initiale et stockée dans le fichier .env de Beetter Home. Cette clé est transmise dans le *header* X-API-Key de chaque requête POST vers /api/push. Coté serveur, la clé est comparée à la valeur attendue avant tout traitement de la trame.

V.B.3. Principaux endpoints

Endpoint	Méthode	Description	Authentification
/api/auth/login	POST	Authentification, retourne un <i>token</i> JWT	Publique
/api/auth/refresh	POST	Renouvellement du <i>token</i> JWT avant expiration	JWT requis
/api/beehives	GET	Liste des ruches avec leur statut courant	JWT requis
/api/beehives/<id>	GET	Détail d'une ruche : mesures récentes, statut, historique d'alertes	JWT requis
/api/beehives/<id>/data	GET	Séries temporelles sur une fenêtre paramétrable	JWT requis
/api/alerts	GET	Journal des alertes actives et historique, filtrable par ruche	JWT requis
/api/push	POST	Réception des trames décodées pushées par Beetter Home	Clé API (<i>header</i>)
/api/remote-config	GET/POST	Configuration de la relation Beetter Home / serveur distant	JWT requis

Tableau V. B-1 Principaux endpoints de l'API REST Beetter Connect

V.B.4. Mécanisme de push Beetter Home vers serveur

Beetter Home envoie périodiquement les données accumulées via POST /api/push. Chaque requête contient un paquet JSON incluant l'identifiant de la ruche, les mesures environnementales et les coefficients MFCC reçus depuis Beetter Hive. Le serveur décode ce paquet, l'écrit dans InfluxDB et met à jour l'état courant dans PostgreSQL.



La résilience face aux interruptions réseau est gérée côté Beetter Home via un mécanisme de file d'attente persistante. Lorsqu'une requête push échoue — que ce soit en raison d'une indisponibilité du serveur, d'une coupure réseau ou d'un délai dépassé — la trame n'est pas perdue : elle est conservée dans une file locale et une nouvelle tentative est planifiée lors du prochain cycle d'APScheduler. Ce mécanisme de *retry* garantit qu'aucune mesure n'est définitivement perdue, même lors d'une interruption prolongée, et que l'historique reste continu et exploitable une fois la connexion rétablie.

V. C. Application mobile Beetter

L'application mobile Beetter est développée avec Flutter, *framework* open-source de Google basé sur le langage Dart. Flutter compile nativement pour Android et iOS depuis une base de code unique, ce qui évite de maintenir deux implémentations séparées. L'application a été développée et validée sur Android. L'architecture retenue ne nécessite aucune modification spécifique pour une publication iOS : seule la procédure de signature et de distribution diffère entre les deux plateformes.

V.C.1. Architecture technique

Librairie	Rôle
Flutter	<i>Framework</i> de développement mobile multiplateforme (Android / iOS) depuis une base de code unique
Riverpod	Gestion d'état réactive : providers pour les données des ruches, l'état d'authentification et les alertes
Dio	Client HTTP pour les appels à l'API REST, avec injection automatique du <i>token</i> JWT via intercepteur
GoRouter	Navigation déclarative avec gestion des routes protégées (redirect si non authentifié)
FL_chart	Génération des graphes interactifs de mesures (température, humidité, fréquence, RMS)

Tableau V. C-1 Stack technique de l'application mobile Beetter

L'application communique exclusivement avec Beetter Connect, jamais directement avec Beetter Home. Ce choix architectural est délibéré et présente plusieurs avantages. D'abord, il simplifie la topologie réseau : l'application n'a besoin de connaître qu'une seule adresse de serveur, quel que soit le nombre de ruchers supervisés. Ensuite, il centralise la sécurité : l'authentification, le contrôle d'accès et le chiffrement ne sont gérés qu'à un seul endroit. Enfin, il préserve l'isolation du réseau local du rucher : Beetter Home n'est jamais exposé directement sur internet, ce qui réduit la surface d'attaque et protège l'installation de l'apiculteur.



V.C.2. Ecrans et fonctionnalités

Ecran	Fonctionnalités
<i>Login</i>	Saisie de l'URL du serveur, identifiant et mot de passe. Obtention et stockage du <i>token</i> JWT.
<i>Dashboard</i>	Liste de toutes les ruches avec statut coloré (Calme / Avertissement / Critique / Sans données), pull-to-refresh.
Détail ruche	Graphes navigables de température, humidité, fréquence et amplitude sonore. Fenêtres temporelles : 1h, 6h, 24h, 7j.
Alertes	Deux onglets : Actives (alertes non résolues) et Historique (journal complet). Indicateurs colorés par gravité.
Paramètres	Configuration de l'URL du serveur Beetter Connect, déconnexion, informations de version.

Tableau V. C-2 Ecrans de l'application mobile

V.C.3. Authentification JWT côté mobile

Au premier lancement, l'utilisateur renseigne l'URL de Beetter Connect dans l'écran paramètres, puis s'authentifie. Le *token* JWT retourné est injecté automatiquement dans toutes les requêtes Dio via un intercepteur dédié. Si le *token* expire, GoRouter redirige vers le *login*.

V.C.4. Alertes push intelligentes

Le système prévoit la réception de notifications push sur le *smartphone*, déclenchées dès qu'une ruche passe en état Critique ou que l'IA détecte un événement comportemental. L'apiculteur est ainsi alerté en temps réel sans avoir à ouvrir l'application. Ce mécanisme sera finalisé lors du déploiement cloud.

V. D. Déploiement et perspectives cloud

L'architecture de Beetter Connect a été conçue pour être déployée sur un serveur cloud, même si la contrainte de temps a conduit à valider la chaîne complète en réseau local d'abord.

V.D.1. Migration vers un VPS cloud

Le passage en production nécessite trois étapes :

- Déploiement du Docker Compose sur un VPS (OVH Cloud ou équivalent), avec IP publique fixe et nom de domaine.
- Activation de HTTPS via un certificat Let's Encrypt pour chiffrer tous les échanges.



- Mise à jour de l'URL serveur dans l'application mobile et dans chaque Beetter Home déployé.

Une fois effectuée, l'apiculteur peut accéder à ses ruches depuis n'importe où sans contrainte réseau.

V.D.2. Architecture multi-ruchers

Plusieurs Beetter Home peuvent pousser vers une instance Beetter Connect unique. Chaque ruche est identifiée par son `hive_id`, et le serveur gère déjà les données multi-sources sans modification architecturale. L'apiculteur dispose d'une vue consolidée de toute sa flotte.

V.D.3. Scalabilité et résilience

- Sauvegarde automatique quotidienne des volumes Docker (InfluxDB et PostgreSQL).
- File d'attente côté Beetter Home pour tamponner les données lors des indisponibilités serveur.
- Monitoring de l'uptime et alertes automatiques en cas de panne.



VI. Bilan et Conclusion

Cette section dresse un bilan des objectifs fixés en début de projet et évalue leur degré d'atteinte, avant de conclure sur les perspectives ouvertes par le projet Beetter.

VI. A. Atteinte des objectifs fonctionnels

Les objectifs fonctionnels du projet ont été organisés en trois catégories : matériel et captation, plateforme et visualisation, intelligence et détection.

Fonctionnalité	Réalisation
Autonomie énergétique	Panneau solaire + Accumulateur. Fonctionnement 24 h/24 sans réseau électrique.
Captation multi-paramètres	Température, humidité et acoustique intégrés.
Transmission LoRa	Transmission en trame unique longue portée.
Redondance des données	Stockage Micro SD local + synchronisation automatique à la reconnexion.
Dashboard temps réel	Interface Flask accessible via le web et l'application mobile Android.
Alertes proactives	Notifications push sur dépassement de seuils et détection d'anomalies.
Gestion de flotte	Supervision centralisée multi-ruches avec carte géographique.
API tierce	Exposition des données via API REST pour interopérabilité.
Analyse acoustique	Intégrée : extraction MFCC embarquée et chaîne de classification en place
IA et apprentissage	Intégrée, en phase de prototypage : modèle contrastif et <i>Random forest</i> déployés

Tableau VI. A-1 Bilan de réalisation des objectifs fonctionnels

VI. B. Valeur apportée & Business plan

Beetter répond concrètement à la problématique initiale : permettre la surveillance distante d'une flotte de ruches sans intervention physique, tout en préservant la santé des colonies. Avec un coût estimé à 200 € par module Beetter Hive, la solution se positionne comme une alternative



accessible aux solutions propriétaires existantes, dont le coût dépasse généralement plusieurs centaines d'euros.

Le coût unitaire de Beetter Hive est estimé à 129,95 € (Annexe VII. E.), incluant les composants électroniques, le PCB, la batterie, le panneau solaire et le boîtier. À ce tarif s'ajoute la passerelle Beetter Home (Raspberry Pi + module LoRa), amortissable sur plusieurs ruches. À titre de comparaison, les solutions commerciales équivalentes telles que Mellisphera ou Beep (Pays-Bas) dépassent généralement 400 € par ruche. Même Mellisphera, l'option la plus abordable du marché, atteint déjà environ 350 € sans la fonctionnalité d'analyse acoustique prédictive – que Beetter propose pour 200 €. Beetter se positionne ainsi comme la solution la plus accessible du marché, avec un rapport fonctionnalités/prix supérieur.

Un modèle économique est envisagé, et deux potentiels axes de revenus sont identifiés :

- Vente matérielle : commercialisation du kit Beetter Hive + Beetter Home à l'unité ou par lot. Marge brute estimée à 30-40 % sur le coût de fabrication (*BOM : Bill of Materials*).
- Abonnement SaaS (*Software as a Service*) : accès aux fonctionnalités avancées (alertes prédictives, historique long terme, API tierce) via un abonnement mensuel envisagé entre 5 et 15 €/mois/rucher.

La France compte 57 500 apiculteurs et 1,45 million de ruches. En ciblant en priorité les apiculteurs semi-professionnels (10 à 200 ruches, soit environ 29 % du marché), le potentiel adressable immédiat représente environ 16 700 apiculteurs. Avec un taux d'adoption de 5 % à horizon 3 ans, cela représente un parc installé de 835 ruchers, soit un chiffre d'affaires potentiel de l'ordre de 250 000 à 500 000 € sur les ventes matérielles, complété par les abonnements récurrents.

Ce business plan doit évoluer, avec une étude de marché à approfondir.

VI. C. Difficultés rencontrées

VI.C.1. Choix des matériaux

A l'origine, Beetter Hive devait être construit avec des plaques de MDF 3 mm découpées et gravées au laser, mais cette solution craignait de ne pas supporter les conditions réelles (humidité et chaleur constante) et le poids des composants. De plus, aucun moyen n'a été trouvé pour garantir proprement la rigidité du boîtier.

De ce fait, la conception du boîtier a été repensée avant d'acheter les matériaux : une armature en tasseaux de bois massif pour assurer la rigidité de l'ensemble (comme une charpente) et des panneaux de polystyrène pour recouvrir les faces.

Ensuite, il a fallu choisir en magasin les plaques de polystyrène et les tasseaux en fonction de leurs dimensions, car nous avons une contrainte d'épaisseur : ne pas dépasser les 25 mm d'un cadre standard Dadant. En considérant les dimensions des plaques de polystyrène de 2,5 mm

(compromis entre finesse et rigidité), des tasseaux de 20 mm de côté auraient été nécessaires, dimensions introuvables en magasin. Il a donc fallu se rabattre sur des tasseaux de 22 mm de côté, qui, selon la conception retenue, peuvent seuls – sans la surépaisseur des plaques – rentrer dans la crémaillère Dadant. Une fois installé dans la ruche, le boîtier est donc 2 mm plus épais, ce qui n'est en réalité pas critique puisqu'entre eux, les cadres sont séparés de 10 mm. Les abeilles ont toujours suffisamment d'espace pour circuler et le cadre a pu être construit.



Figure VI. C-1 Crémaillère Dadant

VI.C.2. Choix de la forme des données audio pour répondre aux contraintes techniques

Pour étudier l'acoustique mesurée par Beetter Hive, l'approche initiale s'est orientée vers la transformation de Fourier à court terme (STFT), voire les données audio brutes pour ensuite les envoyer pour l'analyse sonore. Cependant, des difficultés imprévues qui ont modifié la forme des données sont rapidement apparues.

Pour la transmission des données audio depuis les capteurs de la ruche, le protocole LoRa ainsi que la limite de puissance de l'ESP32 imposent des contraintes fortes sur la taille des trames envoyées : la bande passante est très limitée, et envoyer un signal audio brut ou même un spectre fréquentiel complet ou compressé issu d'une transformée de Fourier classique était tout simplement impossible. Le choix s'est donc porté sur la mesure de coefficients MFCC, une représentation compacte du contenu sonore qui résume l'essentiel de l'information acoustique en un petit nombre de valeurs. Ce compromis entre richesse de l'information et légèreté de la trame a guidé la conception du modèle d'intelligence artificielle, qui a été conçu par la suite pour prendre ces coefficients comme entrée, plutôt qu'un signal audio traditionnel.



VI.C.3. Qualité des données audio et entraînement du modèle de Forêt aléatoire

L'entraînement du modèle de détection basé sur la forêt aléatoire a rapidement été freiné par la faible qualité des enregistrements audio disponibles en ligne. Les bases de données publiques censées représenter des événements apicoles spécifiques – comme les attaques de frelons ou les essaimages – présentaient des conditions d'enregistrement très hétérogènes : bruits de fond importants, niveaux sonores incohérents d'un fichier à l'autre, ou encore des enregistrements trop courts pour être représentatifs. Il a donc fallu intervenir manuellement sur ces données, en appliquant des traitements de nettoyage et de normalisation pour tenter de les rendre exploitables. Malgré ces efforts, la variabilité intrinsèque de ces enregistrements a limité la robustesse du modèle obtenu, ce qui a partiellement contribué à remettre en question cette approche par la suite.

VI.C.4. Adaptation du modèle face aux contraintes du terrain

Au départ, l'objectif était d'entraîner un modèle capable de reconnaître six états distincts de la ruche, allant de l'état normal jusqu'à des événements critiques comme une attaque de frelons ou un essaimage. Cependant, la réalité du terrain a imposé plusieurs compromis importants.

Le premier problème rencontré était lié à la qualité des données collectées. Certains capteurs ont connu des pannes prolongées, introduisant des valeurs erronées qui perturbaient l'apprentissage du modèle. Une fois ces données nettoyées et corrigées, le modèle a montré une bien meilleure capacité à organiser et distinguer les différentes situations vécues par la ruche.

Le second obstacle était l'absence de données étiquetées, c'est-à-dire des données pour lesquelles on sait avec certitude quel événement s'est produit. Sans cela, il est impossible d'apprendre au modèle à reconnaître des situations spécifiques. Les outils initialement prévus pour étiqueter automatiquement les données se sont révélés incompatibles avec les données réellement produites par les capteurs, rendant cette approche inutilisable.

Face à ces contraintes, la stratégie a été revue et simplifiée. Plutôt que de chercher à identifier précisément le type d'événement, le modèle a été redirigé vers une question plus simple mais tout aussi utile : la ruche est-elle dans un état normal ou anormal ? Pour étiqueter les données sans avoir accès à des événements connus, une approche visuelle et semi-manuelle a été adoptée : le modèle regroupe lui-même les données en *clusters* selon leurs similarités, et l'opérateur identifie ensuite lesquels correspondent à des comportements inhabituels en se basant sur les statistiques des capteurs et les périodes concernées. C'est une solution pragmatique qui permet d'aboutir à un résultat exploitable malgré l'absence de données de référence.



Figure VI. C-2 Embedding space avec numérotation des différents clusters

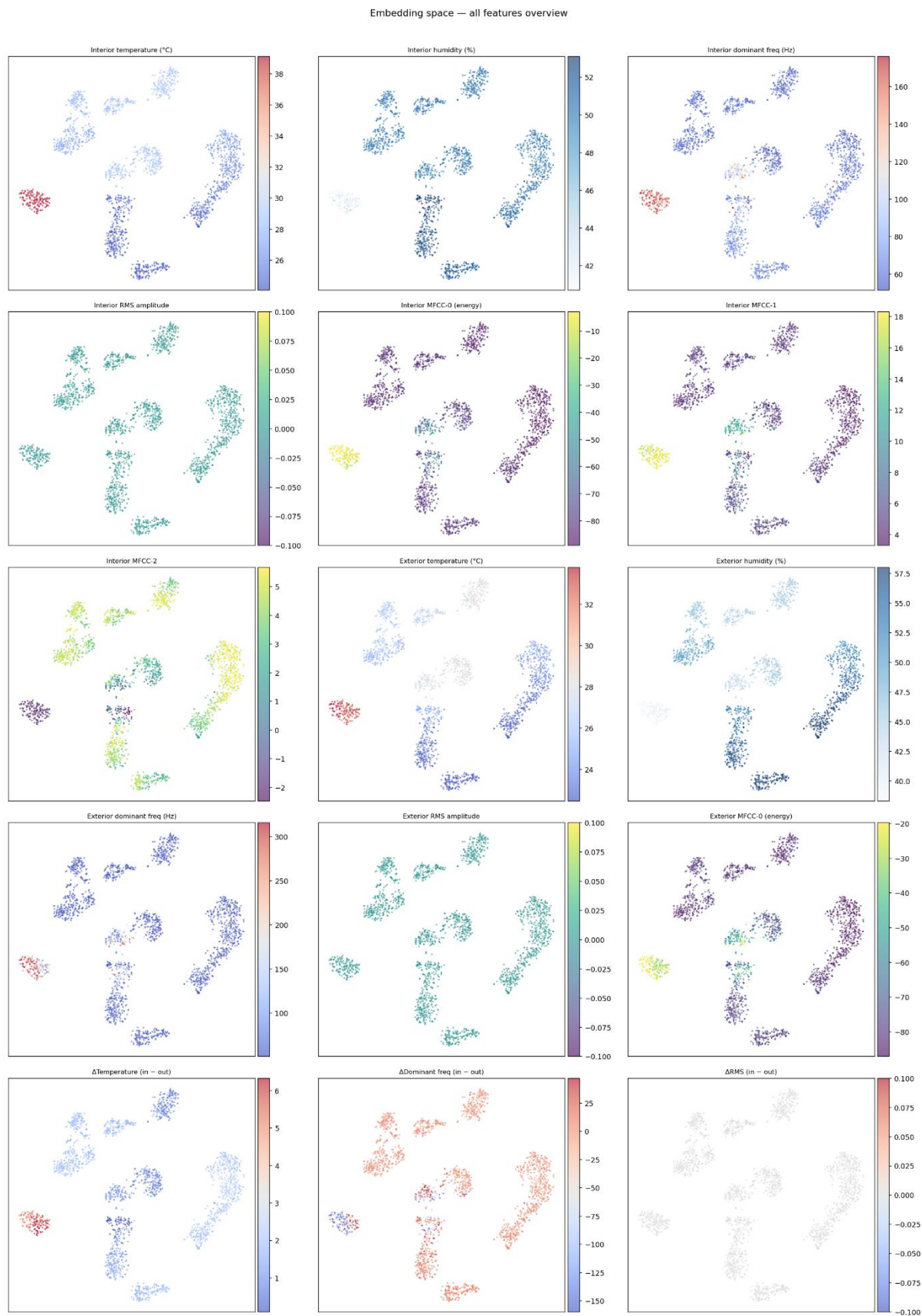


Figure VI. C-3 Graduation des points de l'embedding space en fonction de l'intensité relative à chaque features



L'analyse des *clusters* indique que les groupes 1 et 2 correspondent vraisemblablement à des anomalies, car il y a soit une très forte température intérieure, soit une énergie importante (Coefficient MFCC 0-1), soit une fréquence anormale/distincte.

En conclusion, le modèle final atteint un taux de détection des anomalies de 94 %, ce qui est acceptable et nous semble correct. Nous avons su nous adapter aux contraintes et revers de la mise en situation réelle du module, et avons aménagé les outils et paramètres pour y remédier, en trouvant une alternative intéressante et pragmatique.

VI. D. Perspectives d'évolution

Les axes de développement identifiés pour les versions futures de Beetter incluent :

- L'intégration d'algorithmes d'apprentissage automatique pour affiner la reconnaissance acoustique.
- L'ajout de capteurs complémentaires (poids de la ruche, qualité de l'air).
- Le développement d'un réseau collaboratif permettant aux apiculteurs de partager et de comparer leurs données.
- La démarche de certification en vue d'une éventuelle commercialisation.

VI. E. Ressources numériques et démonstrateur

L'intégralité du code source ainsi que la documentation technique de déploiement, sont versionnés et accessibles sur le dépôt GitHub du projet : <https://github.com/Charlox29/Beetter>

Le démonstrateur fonctionnel, incluant le tableau de bord en temps réel et la cartographie de la flotte de ruches, est exploitable en production à l'adresse suivante : <https://beetter.fr>

VI. F. En conditions réelles chez un apiculteur

Tout au long de la période projet, l'équipe a été en contact avec Alexandra BUTZBACH, apicultrice amateur.

Au tout début, elle a renseigné l'équipe sur les besoins réels des apiculteurs amateurs et professionnels, et les approches ont légèrement changé : nous avons pu étendre notre étude sur l'état de santé des ruches à la détection et prédiction d'essaimage (ce qui n'existe actuellement pas sur le marché).

A force d'échanges avec elle, nous avons aussi pu comprendre comment une ruche fonctionnait, du point de vue des colonies, mais aussi du point de vue physique. Rapidement, elle a validé notre idée de remplacer un cadre usuel par un Beetter Hive, et des mesures ont été prises sur

des ruches format Dadant pour nous rapprocher le plus possible des dimensions standardisées de l'industrie apicole.

Le point culminant de ces échanges a été la mise en conditions réelles du module Beetter Hive et Beetter Home. Le déploiement des deux boîtiers s'est effectué sur site chez l'apicultrice ; pour cette expérience, Beetter Hive a été placé dans une colonie productrice qui venait de trouver une reine.



Figure VI. F-1 Colonie et sa reine

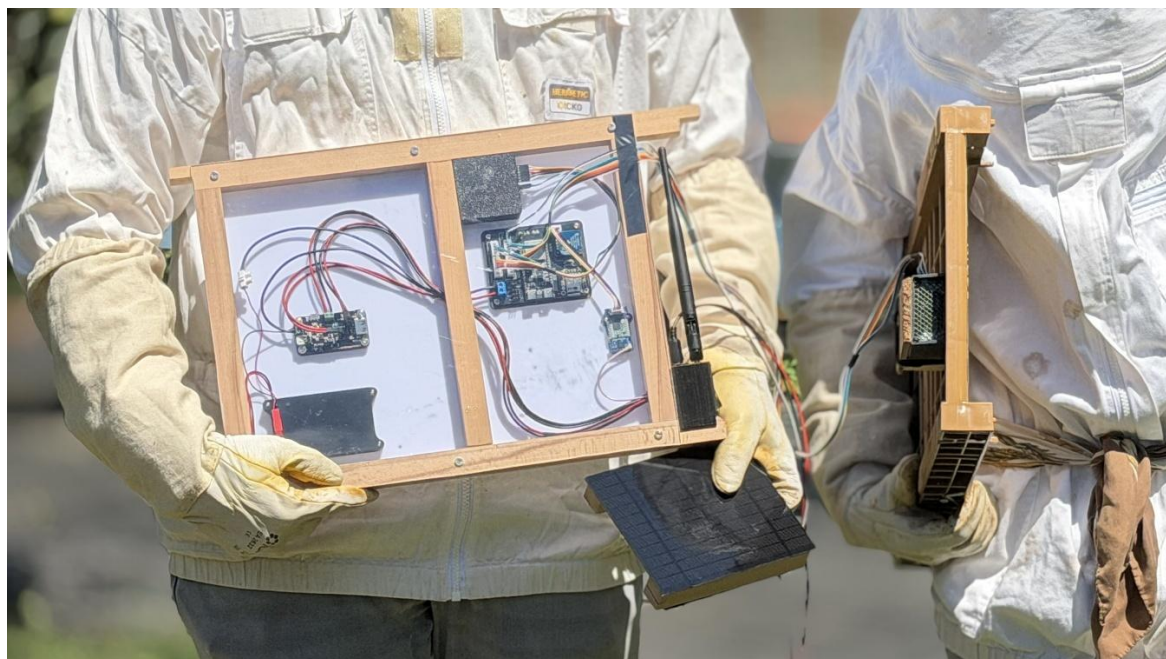


Figure VI. F-2 Modules prêts à être installés sur la ruche expérimentale



Figure VI. F-3 Installation de Beetter Hive dans la ruche



Figure VI. F-4 Vue du dessus de la ruche comportant Beetter Hive

Beetter Home a été configuré avec Raspberry Pi Connect, permettant un accès depuis n'importe où à l'interface locale et aux relevés.

L'expérience a duré 6 jours de canicule pendant lesquels des problèmes sont apparus. Un premier orage a court-circuité le bus I2C du capteur SHT40 extérieur – et parallèlement celui de l'horloge temps réel – causant une mise à 0 des données de température et d'humidité extérieures après un court-circuit supplémentaire entre l'alimentation, Beetter Hive a cessé d'émettre le temps de la pluie. Puis, les relevés envoyés au Beetter Home ont été horodatés en 2031 – à cause du court-circuit I2C – causant l'absence de nouveaux relevés en direct sur le tableau de bord. Pour résoudre temporairement le problème, nous avons mis à jour le programme Python qui recevait les trames LoRa en rajoutant une option sur le choix d'enregistrement des données : soit utiliser l'horodatage transmis par LoRa (alors altéré), soit utiliser l'horodatage direct du Raspberry Pi. Cependant, les données du SHT40 extérieur restaient à 0, donc nous sommes venus visiter la ruche pendant l'expérience pour inspecter les composants, et il a semblé que le capteur n'a pas tenu au court-circuit – hypothèse confirmée en ramenant Beetter Hive lorsque le remplacement du capteur a résolu la mesure des données.

Une autre panne nous est survenue avant la fin de l'expérience ; sans corrélation avec la météo, le module a cette fois arrêté d'émettre. C'est à la fin de l'expérience, en ressortant le boîtier de sa phase expérimentale, et en redémarrant son alimentation, l'émission et la réception des données ont pu reprendre.

A travers les difficultés rencontrées, le projet a pu évoluer ; les pièces du boîtier extérieur ont été redessinées et réimprimées pour permettre à l'eau de s'écouler en cas de stagnation. La couleur a aussi changé – de noir à blanc – car la température avoisinait les 50 °C.

En plus de ses précieux conseils, Alexandra a réalisé une démonstration du marquage de la reine. Pour un apiculteur, cette tâche est indispensable pour retrouver la reine lorsque la colonie se repeuple ; si la reine n'est pas retrouvée, cela peut signifier qu'elle a quitté la ruche pour essaimer.



Figure VI. F-5 Marquage de la reine



Figure VI. F-6 Reine marquée

Une fois marquée, la reine est placée dans une cage dont l'ouverture est bouchée par une sorte de sucre ; le temps qu'elle et les autres abeilles mangent le sucre pour la libérer, les ouvrières s'habituent à l'odeur de la peinture et ne la rejettent plus.

En conclusion, le projet Beetter démontre la faisabilité technique et économique d'une solution IoT de monitoring apicole accessible. Il constitue un premier jalon vers une apiculture numérique, au service de la préservation des colonies et de la biodiversité.



VII. Annexes

Les annexes suivantes complètent le corps du rapport avec des informations techniques détaillées et les références bibliographiques. Elles permettent au lecteur souhaitant approfondir certains aspects de disposer d'un référentiel complet sans alourdir la lecture principale.

VII. A. Biologie apicole

VII.A.1. Cycle de vie des abeilles

Chaque abeille traverse quatre stades de développement distincts : l'œuf, la larve, la puppe (ou nymphe) et l'adulte (imago). Le destin d'une abeille femelle – ouvrière ou reine – dépend exclusivement de son alimentation larvaire : les larves nourries en continu à la gelée royale dans une cellule royale deviennent reines ; les autres deviennent ouvrières. Un œuf non fécondé (haploïde) donne naissance à un faux-bourdon^[3].

Stade de croissance	Reine (Femelle)	Ouvrière (Femelle)	Faux-bourdon (Mâle)
Stade Œuf	3 jours	3 jours	3 jours
Stade Larve (Cellule ouverte)	5,5 jours	6 jours	6,5 jours
Operculation (Cellule fermée)	Au 8ème jour	Au 9ème jour	Au 10ème jour
Stade Puppe / Nymphe	7,5 jours	12 jours	14 jours
Durée totale (Naissance)	16 jours	21 jours	24 jours

Tableau VII. A-1 Durée des stades de développement selon la caste

VII.A.2. Les métiers de l'ouvrière

Une fois sortie de sa cellule, l'ouvrière ne butine pas immédiatement. Sa vie d'adulte est une succession de métiers dictés par son âge et le développement de ses glandes :

Période	Rôle	Tâches principales
Jours 1–3	Nettoyeuse	Nettoie les alvéoles vides, élimine débris et opercules pour préparer les cellules à la prochaine ponte.
Jours 3–10	Nourrice	Produit de la gelée royale via ses glandes hypopharyngiennes. Nourrit les larves et la reine.



Période	Rôle	Tâches principales
Jours 10–18	Cirière / Bâtisseuse	Sécrète de la cire via ses 8 glandes abdominales. Construit et répare les rayons hexagonaux.
Jours 14–18	Magasinière	Réceptionne le nectar des butineuses, le transforme en miel, opercule les cellules pleines.
Jours 12–18	Ventileuse	Bat des ailes pour réguler la température (35 °C) et l'humidité, et déshydrater le nectar.
Jours 18–21	Gardienne	Contrôle l'entrée de la ruche, identifie les abeilles de la colonie par odeur, repousse les intrus.
Jour 21 → mort	Butineuse	Collecte nectar, pollen, eau, propolis. Vole jusqu'à 5 km. Communique les sources par la danse.

Tableau VII. A-2 Métiers successifs de l'ouvrière selon son âge

VII. B. Référentiel de monitoring

VII.B.1. Valeurs de référence^{[5][6]}

Paramètre	Valeur	Signification
Température couvain	33 – 36 °C	Zone optimale pour le développement des larves et nymphes (couvain sténotherme)
Température idéale	35 °C	Valeur cible au centre du nid à couvain pour une colonie forte
Température létale	> 39 °C	Au-delà, les abeilles meurent ; entre 36 - 39°C, risque de malformation des ailes
Humidité relative interne	50 – 60 %	Plage optimale pour la santé du couvain et la conservation du miel
Humidité absolue interne	~17 g/m ³	Indicateur recommandé : toujours supérieur à l'extérieur, reflet de l'activité des abeilles
Humidité production miel	Jusqu'à 95 %	Temporairement élevée lors de l'évaporation du nectar (teneur cible ~16 %)
Grappe hivernale (centre)	> 10 °C	En dessous de 7°C le corps de l'abeille se paralyse ; < 10°C capacité de vol affaiblie
Reprise de ponte (hiver)	+ 0,1°C/jour	Augmentation progressive en jan–mars indique la reprise d'activité

Tableau VII. B-1 Valeurs environnementales de référence



VII.B.2. Santé générale de la colonie

La capacité de la colonie à maintenir sa température interne stable et avec de faibles fluctuations est le meilleur indicateur de sa santé. Une colonie forte thermorégule en permanence autour de 35 °C avec peu de variation et une colonie affaiblie laisse la température interne suivre les conditions extérieures.

- Colonie saine et forte : température stable 33–36 °C, fluctuations journalières faibles, indépendantes de la météo extérieure.
- Colonie en déclin : la température interne fluctue davantage et se rapproche des températures ambiantes.
- Ruche morte ou sans abeilles : température interne = température externe.
- Absence de couvain : sans couvain actif, les abeilles n'ont plus d'incitation à thermoréguler donc la température suit les conditions ambiantes.
- Sur l'humidité : une humidité absolue intérieure qui converge vers les valeurs extérieures signale un affaiblissement de la colonie, car les abeilles ne produisent plus suffisamment d'activité pour la maintenir élevée.

VII.B.3. Essaimage

- Agitation de la colonie se traduisant par une perte de contrôle de la régulation thermique, détectable quelques jours avant l'événement.
- Hausse soudaine et atypique de température interne, non corrélée à la météo extérieure.

VII.B.4. Présence et activité de la reine

- Température > 30 °C en dehors de la zone de couvain : la ponte s'étend, la population augmente.
- Absence de reine : la thermorégulation se dégrade progressivement par manque de nouvelle génération – se manifeste d'abord par une augmentation des fluctuations quotidiennes avant une chute des valeurs moyennes.

VII.B.5. Risques liés à l'humidité

- Humidité absolue anormalement élevée : risque de champignons, fermentation du miel, propagation de maladies.
- Humidité absolue qui se rapproche des valeurs extérieures : signal de colonie affaiblie ou de perte d'activité.



- Humidité trop basse (régions sèches) : risque de dessèchement du couvain.
- Note : l'humidité relative seule peut être trompeuse dans la ruche (température élevée = RH apparemment basse). Préférer l'humidité absolue pour les alertes.

VII.B.6. Intoxication aux pesticides

- Chute brutale des fluctuations thermiques normales (la colonie cesse de thermoréguler activement) pouvant précéder une chute de température. À croiser avec d'autres paramètres (poids, acoustique).

VII. C. Acoustique apicole

VII.C.1. Acoustique apicole : référentiel des fréquences

L'apiculture acoustique^{[4][5]} consiste à écouter et analyser les sons produits par la ruche (bourdonnements, vibrations, fréquences spécifiques) afin d'en évaluer l'état de santé, l'activité ou les intentions, sans ouvrir la ruche ni déranger la colonie. Le bourdonnement de l'abeille résulte de deux phénomènes combinés : le battement des ailes à environ 230 fois par seconde, et les vibrations du thorax dues à la contraction musculaire.

Le tableau suivant recense les signatures acoustiques observées selon les comportements de la colonie, et constitue la base de données de référence pour l'entraînement des algorithmes de classification du projet :

Comportement / Situation	Fréquence dominante (Hz)	Caractéristiques acoustiques	Interprétation / Alerte
Colonie calme, équilibrée ^{[4][7]}	200 – 250	Bourdonnement grave, régulier, continu	État normal, colonie saine
Butinage actif (vol d'ouvrière) ^[7]	220 – 260	Fréquence stable, tonalité constante	Activité normale de collecte
Ventilation de la ruche ^[4]	200 – 230	Son rythmé, collectif, synchronisé	Régulation thermique ou hydrique
Stress léger (chaleur, manque de ressources) ^{[5][6]}	300 – 400	Modulations rapides, irrégularités de timbre	Surveillance accrue recommandée



Comportement / Situation	Fréquence dominante (Hz)	Caractéristiques acoustiques	Interprétation / Alerte
Agitation / Danger détecté ^[5]	400 – 500	Bourdonnement fort, irrégulier, tonalité montante	Intervention possible de l'apiculteur
Essaimage imminent (chant de la reine) ^{[8][9][10]}	350 – 500	Sons intermittents aigus, rafales courtes répétées	Alerte : essaimage à prévoir sous 24–72 h
Orphelinage (absence de reine) ^{[5][6]}	300 – 450	Bourdonnement anxieux, plus aigu, instable	Vérifier présence de la reine
Intrusion de prédateur (frelon) ^[11]	400 – 600	Son strident, saccadé, montée rapide de fréquence	Présence de prédateur – observer
Ruche silencieuse (hors repos nocturne) ^[4]	< 100 ou silence	Absence de bourdonnement perceptible	Colonie en danger ou morte
Chant de la reine vierge ^{[8][9]}	~500 (pic aigu)	Son bref, aigu, intermittent (tooting/quacking)	Cellule royale éclore, nouvelle reine en lice

Tableau VII. C-1 Signatures acoustiques de référence par comportement de la colonie

Note de lecture : Ces valeurs sont des plages indicatives issues de la littérature apicole. La fréquence exacte varie selon la race d'abeille, la saison, la température ambiante et les conditions de la ruche. L'analyse FFT/MFCC permet de décomposer le signal sonore en composantes fréquentielles pour identifier ces signatures.

VII.C.2. Comparatif : abeilles, bourdons, frelons, guêpes et taons

La capacité à distinguer acoustiquement les espèces est un enjeu clé pour le monitoring apicole. Le tableau ci-dessous compare les principales espèces susceptibles d'être détectées à proximité ou à l'intérieur d'une ruche, selon leur morphologie, leur fréquence de vol, leur comportement et leur niveau de dangerosité pour la colonie.

Insecte	Taille (mm)	Fréquence de vol (Hz)	Comportement	Dangerosité
Abeille domestique (Apis mellifera) ^[7]	12 – 15	230 – 270	Sociale, non agressive sauf menace directe. Meurt après avoir piqué. Pollinisatrice essentielle.	Faible



Insecte	Taille (mm)	Fréquence de vol (Hz)	Comportement	Dangerosité
Bourdon (Bombus sp.) ^[12]	15 – 25	130 – 200	Social mais colonies petites (< 200). Peu agressif, rarement pique. Vol lent et lourd, bourdonnement grave et sonore.	Très faible
Guêpe commune (Vespula vulgaris) ^[4]	12 – 17	300 – 400	Sociale, charognarde et prédatrice. Agressive en fin de saison. Peut piquer plusieurs fois. Attire aux sucres et viandes.	Modéré
Frelon européen (Vespa crabro) ^[4]	25 – 35	110 – 150	Social, prédateur d'insectes dont les abeilles. Moins agressif que le frelon asiatique. Vol nocturne possible. Nuisible à la ruche mais impact limité.	Modéré
Frelon asiatique (Vespa velutina) ^[11]	25 – 30	140 – 185	Prédateur spécialisé des abeilles mellifères. Vole en stationnaire devant les ruches pour capturer les butineuses. Chasse en groupe intensif et prolongé.	Très élevé
Taon (Tabanus sp.) ^[4]	15 – 25	100 – 160	Solitaire, hématophage (se nourrit de sang). Ne s'attaque pas aux ruches. Nuisance directe à l'apiculteur par ses piqûres douloureuses.	Faible (humain)

Tableau VII. C-2 Fréquences de vol et caractéristiques acoustiques par espèce

Distinction acoustique clé : Le frelon asiatique (*Vespa velutina*) émet une fréquence de vol légèrement plus élevée (140–185 Hz) que le frelon européen (*Vespa crabro*, 110–150 Hz)^[11], plus gros et au bourdonnement plus grave, proche de celui du bourdon. Les plages se recouvrant partiellement, la discrimination automatique ne repose pas sur la seule fréquence dominante mais sur l'empreinte timbrale complète (MFCC) et sur la signature de vol stationnaire du frelon asiatique devant l'entrée de la ruche^{[4][5]}, qui génère des vibrations basse fréquence détectables par le microphone interne.

VII. D. L'Essaimage : phénomène biologique et enjeux de détection

VII.D.1. Définition et mécanisme^[3]

L'essaimage est le processus naturel par lequel une colonie d'abeilles se reproduit en se divisant. Lorsque la population atteint un seuil critique, généralement en fin de printemps, la vieille reine quitte la ruche accompagnée de 30 à 60 % des ouvrières pour fonder une nouvelle colonie ailleurs. Ce départ massif et brutal constitue l'événement le plus redouté de l'apiculteur : il peut



engendrer une perte de production de 40 à 70 % sur la saison en cours, et survient en quelques minutes, sans signe visible préalable pour un observateur non équipé.

L'essaimage n'est pas un dysfonctionnement : c'est un comportement instinctif de survie de l'espèce. C'est précisément pourquoi il est impossible à éliminer, et pourquoi sa détection anticipée est la seule réponse efficace.

L'essaimage suit une progression en cinq phases distinctes, chacune présentant des signatures mesurables par les capteurs de Beetter :

Phase	Délai avant départ	Signal thermique	Signal acoustique
Construction des cellules royales	J - 15	Stable	Légère agitation 250–300 Hz
Agitation croissante	J - 5	Hausse anormale +1–2 °C/h	Modulations irrégulières 300–400 Hz
Chant de la reine vierge	J - 1	Forte instabilité thermique	Sons intermittents aigus 350–500 Hz
Départ de l'essaim	Jour J	Chute brutale de température	Bourdonnement massif puis silence
Réorganisation	J + 2	Reprise progressive	Retour à une signature calme

Tableau VII. D-1 Phases d'essaimage d'une colonie

VII.D.2. Pourquoi l'acoustique est le signal clé^{[8][9][10]}

Les capteurs de température et d'humidité seuls ne suffisent pas à détecter un essaimage imminent avec fiabilité : une journée chaude, une ruche en pleine miellée ou une forte activité de ventilation peuvent produire des signatures thermiques similaires.

Deux phénomènes sonores sont spécifiques à la préparation d'un essaimage :

- Le *piping*/chant de la reine (*tooting* et *quacking*) : la reine vierge éclore émet des sons brefs et aigus (pic vers 500 Hz) pour signaler sa présence aux ouvrières^{[8][9]}. Ce signal, absent en conditions normales, constitue le marqueur acoustique le plus fiable d'un essaimage imminent^[10].
- L'agitation pré-essaimage : dans les 5 jours précédant le départ, la fréquence dominante du bourdonnement de la colonie monte progressivement de 230 Hz (état calme) vers 350–450 Hz, avec une augmentation du niveau sonore global et des irrégularités de timbre caractéristiques d'une colonie en état de stress collectif.



VII.D.3. Facteurs déclencheurs et contexte saisonnier

Plusieurs facteurs favorisent l'essaimage et permettent d'affiner le modèle de prédiction :

- **Surpopulation** : une ruche de plus de 60 000 individus en période de miellée est statistiquement à risque.
- **Manque d'espace de ponte** : lorsque la reine n'a plus de cellules disponibles, le départ devient inévitable.
- **Saison** : la période à risque maximale s'étend d'**avril à juin** en France métropolitaine, avec un pic en mai.
- **Météo** : les essaimages surviennent préférentiellement lors de journées chaudes et ensoleillées, entre 10h et 14h.

VII.D.4. Impact économique et justification du projet

À l'échelle nationale, en supposant un taux d'essaimage de 15 % des colonies par an et une perte de production moyenne de 50 % par ruche essaimée, on estime la perte économique annuelle pour la filière française à plusieurs dizaines de millions d'euros. Pour un petit apiculteur gérant 10 ruches, un essaimage non détecté représente une perte pouvant dépasser 300 € de miel sur la saison, en plus de la perte d'une partie du cheptel.

Beetter répond directement à cet enjeu : en détectant l'essaimage, il offre à l'apiculteur le temps d'intervenir : ajouter une hausse, capturer un essaim, diviser une colonie et de transformer un événement subi en une opportunité maîtrisée.

VII. E. Documentation projet

VII.E.1. Objectifs fonctionnels détaillés

La liste exhaustive des objectifs fonctionnels identifiés en début de projet est présentée ci-dessous, organisée par domaine.

Matériel et captation

- Prévoir des broches d'extension (pins) supplémentaires sur le PCB
- Assurer le fonctionnement en autonomie énergétique complète
- Mesurer en continu l'environnement intérieur et extérieur de la ruche
- Acquérir et analyser les sons produits par la colonie et l'environnement



- Transmettre l'intégralité des données vers le serveur en une seule trame LoRa
- Conserver les données localement en cas de perte de connexion
- Synchroniser automatiquement l'horloge lors de la reconnexion

Plateforme et visualisation

- Stocker l'historique complet de chaque ruche
- Afficher l'état de chaque ruche en temps réel
- Permettre la navigation dans l'historique sur des plages temporelles personnalisées
- Envoyer des alertes immédiates en cas d'anomalie détectée
- Gérer une flotte de plusieurs ruches depuis une interface unique
- Afficher la position de chaque ruche sur une carte interactive
- Accéder au tableau de bord depuis un *smartphone*
- Exposer les données via une API REST utilisable par des services tiers
- Exporter les données d'une ruche pour analyse externe
- Sécuriser les accès par authentification utilisateur
- Proposer une modification de la langue et du thème de l'interface

Intelligence et détection

- Évaluer automatiquement l'état de santé d'une colonie
- Détecter la présence de prédateurs (frelon asiatique, autres)
- Anticiper un essaimage avant sa survenue
- Détecter une perte de reine (orphelinage)
- Distinguer acoustiquement les bourdons des abeilles ouvrières

VII.E.2. Glossaire des acronymes et termes techniques

Les termes et acronymes suivants sont utilisés dans ce rapport. Conformément aux règles rédactionnelles, chaque acronyme est défini lors de sa première apparition dans le texte, et repris ici pour référence.



Terme	Définition
ESP32-C6	Microcontrôleur (MCU) Wi-Fi / Bluetooth, basse consommation, fabriqué par Espressif Systems. Constitue l'unité de traitement centrale du boîtier Beetter Hive.
IoT	Internet of Things (Internet des Objets) : désigne les objets physiques connectés à internet capables de collecter et d'échanger des données.
LoRa	Long Range : technologie de modulation radio basse consommation permettant des transmissions sur de longues distances (plusieurs kilomètres) avec un faible débit.
PCB	Printed Circuit Board (Circuit Imprimé) : support physique sur lequel sont soudés les composants électroniques.
RTC	Real-Time Clock (Horloge Temps Réel) : module électronique maintenant l'heure courante de façon autonome, même sans alimentation principale.
SHT40	Capteur de température et d'humidité relative fabriqué par Sensirion, référencé SEN0428 dans la gamme DFRobot.
ADA6049	Microphone numérique I2S (Inter-Integrated Circuit Sound), référencé par Adafruit, dédié à l'acquisition acoustique.
RFM95	Module radio LoRa de la gamme HopeRF, permettant les transmissions longue portée bas débit.
LiPo	Lithium Polymère : technologie de batterie rechargeable légère et à haute densité d'énergie.
InfluxDB	Base de données orientée séries temporelles (time-series database), optimisée pour stocker des données horodatées à haute fréquence.
PostgreSQL	Système de gestion de base de données relationnelle (SGBDR) open-source, utilisé ici pour les métadonnées.
Flask	Micro-framework Python pour le développement d'applications web légères et d'API REST (Representational State Transfer).
API	<i>Application Programming Interface</i> : interface exposant des fonctionnalités logicielles accessibles par des services tiers.
VPS	Virtual Private Server (Serveur Privé Virtuel) : serveur mutualisé offrant un environnement d'exécution dédié dans le cloud.
FFT	Fast Fourier Transform (Transformée de Fourier Rapide) : algorithme d'analyse spectrale permettant de décomposer un signal sonore en fréquences.
IHM	Interface Homme-Machine : désigne toute interface permettant à un utilisateur d'interagir avec un système informatique.



Terme	Définition
MFCC	Mel-Frequency Cepstral Coefficients (Coefficients Cepstraux sur l'Échelle de Mel) : représentation compacte du timbre sonore d'un signal audio, calculée à partir de la transformée de Fourier (FFT) et d'un filtrage inspiré de la perception auditive humaine. Utilisés ici pour caractériser la signature acoustique de la ruche et alimenter le modèle de classification des états comportementaux.
Bin (FFT)	Case de fréquence élémentaire produite par la FFT. Le spectre est découpé en <i>bins</i> de largeur constante (ici 3,9 Hz = F_s/N , soit 8 000/2 048) ; le <i>bin</i> de plus forte amplitude indique la fréquence dominante du signal sonore.
RMS	Root Mean Square (valeur efficace) : moyenne quadratique d'un signal – racine de la moyenne des carrés des échantillons. Employée ici comme mesure de l'amplitude (du niveau sonore) d'une fenêtre audio.

Tableau VII. E-1 Glossaire des acronymes et termes techniques

VII.E.3. Nomenclature produit et dépenses

Module	Composants	Prix (€)	Quantité
Beetter Hive	SEN0428 (SHT 40)	6,70	2
	6049 (Micro)	4,25	2
	ESP32-C6-DevKitC-1-N8	7,50	1
	4682 (Micro SD)	3,00	1
	313070001 (Panneau Solaire)	10,30	1
	DFR0559 (Solar Power Manager)	6,80	1
	610 (Prise Barrel Jack)	2,55	1
	PP-050A (Plug Barrel Jack)	1,35	1
	RR812C1121 (Interrupteur)	1,40	1
	Module Grove LoRa 113060006	22,50	1
	Connecteur JST SYP vers fils JSTSYP2P	0,35	1
	Accu LiPo 3,7 Vcc 4000 mAh L805080	19,90	1
	RTC ADA3295	8,30	1
	PCB (un seul utilisé)	10,50	5
	Bois / Polystyrène	22,00	1



Module	Composants	Prix (€)	Quantité
Beetter Home	851 (Cable uFL SMT vers PMA)	3,40	1
	AEACAC195013-S868 (Antenne)	6,70	1
	Raspberry Pi (5) (emprunté)	192,00	1
	Module Grove LoRa 113060006	22,50	1
Nom de domaine Beetter.fr		5,99	1
Divers		5,00	1
Total dépensé		223,94 €	
Coût Beetter Hive seul		129,95 €	
Coût Beetter Home seul		224,60 €	

Tableau VII. E-2 Nomenclature des composants et coût total du projet



VIII. Bibliographie

VIII. A. Références

- [1] FranceAgriMer et Agrex Consulting, « Observatoire de la production de miel, de gelée royale et des autres produits de la ruche – Données 2024 », FranceAgriMer, Montreuil-sous-Bois, France, juill. 2025. [En ligne]. Disponible : https://www.franceagrimer.fr/sites/default/files/2025-07/SYN-API_Observatoire_Miel_et_GeleeRoyale_2024%20.docx.pdf
- [2] Ministère de l'Agriculture et de la Souveraineté Alimentaire, « Fiche filière apiculture », FranceAgriMer, Montreuil-sous-Bois, France, 2026. [En ligne]. Disponible : <https://www.franceagrimer.fr/chiffre-et-analyses-economiques/fiche-filiere-apiculture>
- [3] GDS France, « Données sanitaires et démographiques des colonies d'abeilles en France », Groupements de Défense Sanitaire, Paris, France. [En ligne]. Disponible : <https://www.gdsfrance.org>
- [4] W. H. Kirchner, « Acoustical communication in honeybees », *Apidologie*, vol. 24, no. 3, pp. 297–307, 1993. DOI : 10.1051/apido:19930309
- [5] A. Terenzi, S. Cecchi et S. Spinsante, « On the Importance of the Sound Emitted by Honey Bee Hives », *Veterinary Sciences*, vol. 7, no. 4, p. 168, oct. 2020. DOI : 10.3390/vetsci7040168
- [6] S. Ferrari, M. Silva, M. Guarino et D. Berckmans, « Monitoring of swarming sounds in bee hives for early detection of the swarming period », *Computers and Electronics in Agriculture*, vol. 64, no. 1, pp. 72–77, 2008. DOI : 10.1016/j.compag.2008.05.010
- [7] N. P. Goyal et A. S. Atwal, « Wing Beat Frequencies of *Apis cerana indica* and *Apis mellifera* », *Journal of Apicultural Research*, vol. 16, no. 1, pp. 47–48, 1977. DOI : 10.1080/00218839.1977.11099859
- [8] T. D. Seeley et J. Tautz, « Worker *pip*ing in honey bee swarms and its role in preparing for liftoff », *Journal of Comparative Physiology A*, vol. 187, no. 8, pp. 667–676, 2001. DOI : 10.1007/s00359-001-0243-0
- [9] A. M. Wenner, « Sound production during the waggle dance of the honey bee », *Animal Behaviour*, vol. 10, no. 1–2, pp. 79–95, 1962. DOI : 10.1016/0003-3472(62)90135-5
- [10] A. L. Pierce, L. A. Lewis et S. S. Schneider, « The Use of the Vibration Signal and Worker *Piping* to Influence Queen Behavior during Swarming in Honey Bees, *Apis mellifera* », *Ethology*, vol. 113, no. 3, pp. 267–275, 2007. DOI : 10.1111/j.1439-0310.2006.01314.x
- [11] M. Arca, A. Papachristoforou, F. Mougél, A. Rortais, K. Monceau, O. Bonnard, P. Tardy, D. Thiéry, J.-F. Silvain et G. Arnold, « Defensive behaviour of *Apis mellifera* against *Vespa*



velutina in France : Testing whether European honeybees can develop an effective collective defence against a new predator », Behavioural Processes, vol. 106, pp. 122–129, 2014. DOI : 10.1016/j.beproc.2014.05.002

- [12] B. Joos, P. A. Young et T. M. Casey, « Wingstroke frequency of foraging and hovering bumblebees in relation to morphology and temperature », Physiological Entomology, vol. 16, no. 2, pp. 191–200, 1991. DOI : 10.1111/j.1365-3032.1991.tb00556.x

VIII. B. Autres sources consultées

- Naturapi, « L'apiculture acoustique : surveiller une ruche sans l'ouvrir ». [En ligne]. Disponible : <https://www.naturapi.com/naturapicafe> [consulté juin 2026]
- Apiculture.net, « Le cycle de vie des abeilles ». [En ligne]. Disponible : <https://www.apiculture.net> [consulté juin 2026]
- InterApi, Interprofession des produits de la ruche. [En ligne]. Disponible : <https://interapi.fr>
- INRAE, Institut national de recherche pour l'agriculture, l'alimentation et l'environnement. [En ligne]. Disponible : <https://www.inrae.fr>
- Allo-Frelons. [En ligne]. Disponible : <https://allo-frelons.fr/vibrations-des-abeilles>
- Abeilles & Environnement. [En ligne]. Disponible : <https://www.abeilles-environnement.com>
- W. H. Kirchner, « Signals and Signal Responses in Honeybee Acoustics », in Bee Communication Biology (Lecture Notes), Univ. Würzburg, 1997.
- C. J. Clark, A. M. Mountcastle, E. Mistick et D. O. Elias, « Resonance frequencies of honeybee (Apis mellifera) wings », Journal of Experimental Biology, vol. 220, no. 16, pp. 2887–2895, 2017. DOI : 10.1242/jeb.154609
- D. M. Unwin et S. A. Corbet, « Wingbeat frequency, temperature and body size in bees and flies », Physiological Entomology, vol. 9, no. 1, pp. 115–121, 1984. DOI : 10.1111/j.1365-3032.1984.tb00687.x

VIII. C. Remerciements

Nous souhaitons remercier nos professeurs et nos contacts pour leur aide précieuse, et en particulier :

- Louis GAMBART pour l'infrastructure logicielle ;



- Alexandra BUTZBACH pour le matériel apicole, son implication et son soutien dans notre projet ;
- Olivier FRANÇAIS pour avoir suivi les différentes étapes de notre projet et pour nous avoir conseillé ;
- Quentin ROME pour l'enregistrement de frelons, Laurent BUES pour l'achat des composants ;
- Anne EXERTIER pour le prêt de la salle de projet.

VIII. D. Mention IA

Conformément à un souci de transparence sur l'utilisation de l'intelligence artificielle dans la réalisation de ce projet, l'équipe Beetter précise avoir eu recours à plusieurs outils d'IA générative en appui du travail de conception, de développement et de rédaction.

Les outils suivants ont été utilisés : ChatGPT (OpenAI), Claude (Anthropic), Gemini (Google) et Sci-Bot.

Ces outils ont été mobilisés à titre d'assistance pour :

- La génération et la structuration de documentation technique (commentaires de code, fichiers de configuration, portions rédactionnelles de ce rapport) ;
- L'aide au développement logiciel : génération de code, débogage, optimisation ;
- La recherche de documentation technique et de références bibliographiques.

L'ensemble des contenus produits avec l'aide de ces outils a systématiquement été relu, vérifié et validé par les membres de l'équipe avant son intégration au projet ou au présent rapport. L'intelligence artificielle a ainsi été employée comme un outil de productivité et d'assistance, sans se substituer à la réflexion, à la conception technique ni à la responsabilité scientifique des auteurs, qui restent garants de l'exactitude et de l'intégrité de l'ensemble du contenu produit.